

project

WEEK3 - TUE - YANN

ideas

-audio reactive visual with if-statements -> if sound fast = speed visual higher

coaching

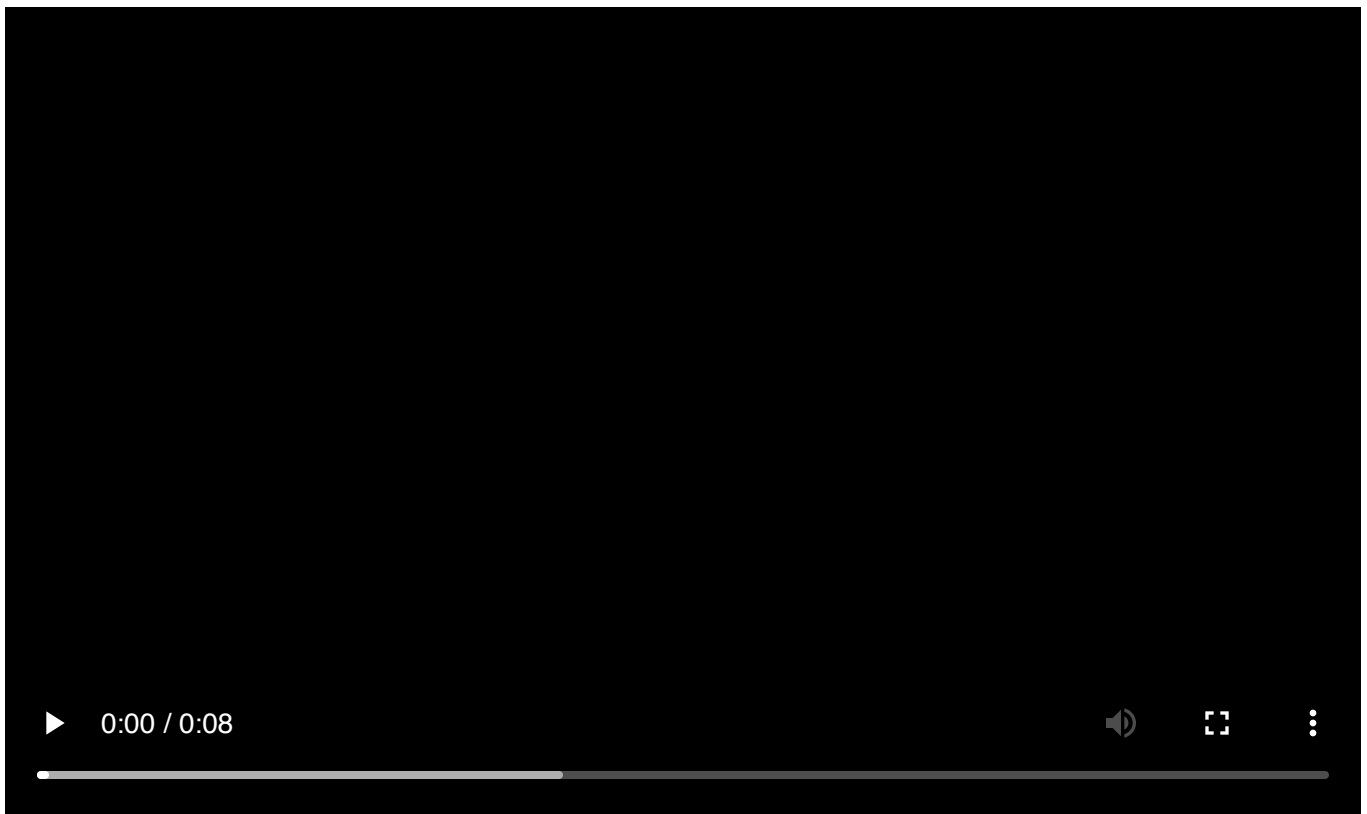
it's okay for "live", just needs to be interactive

p5LIVE audio reactive ->

WEEK3 - WED

experiments

trying to create a system - synthetic



```
let tiles = []; // array für kacheln
let s = 20;

function setup() {
  createCanvas(windowWidth, windowHeight);
```

```

rectMode(CENTER);
noStroke();

for (let y = 0; y < height; y += s) { // grid zeile für zeile

  for (let x = 0; x < width; x += s) { // grid spalte für spalte

    tiles.push({

      x,
      y,

      type: floor(random(6)), // zufällige form (0-5)

      col: random([ // zufällige farbe
        "#9400D3",
        "#32CD32",
        "#b8b0a5",
        "#111111"
      ])

    });

  }
}

function draw() {
  background("#006400");

  for (let t of tiles) { // alle kacheln durchlaufen

    fill(t.col);

    push();

    translate(t.x, t.y);

    // kreis
    if (t.type == 0) {
      ellipse(0, 0, s * 0.9);
    }

    // square
    if (t.type == 1) {

```

```

    rect(0, 0, s * 0.9, s * 0.9);
}

// plus
if (t.type == 2) {

    rect(0, 0, s * 0.25, s); // vertikal
    rect(0, 0, s, s * 0.25); // horizontal
}

// linien
if (t.type == 3) {

    rect(0, -5, s, 3); // obere linie
    rect(0, 0, s, 3); // mittlere linie
    rect(0, 5, s, 3); // untere linie
}

// ring
if (t.type == 4) {

    ellipse(0, 0, s * 0.9); // äusserer kreis

    fill("#050505"); // dunkle Farbe für loch

    ellipse(0, 0, s * 0.4); // innerer kreis
}

// kleine dots
if (t.type == 5) {

    ellipse(-5, -5, 4); // oben links
    ellipse(5, -5, 4); // oben rechts
    ellipse(-5, 5, 4); // unten links
    ellipse(5, 5, 4); // unten rechts
}

pop();

// selten verändern
if (random(1) < 0.002) {

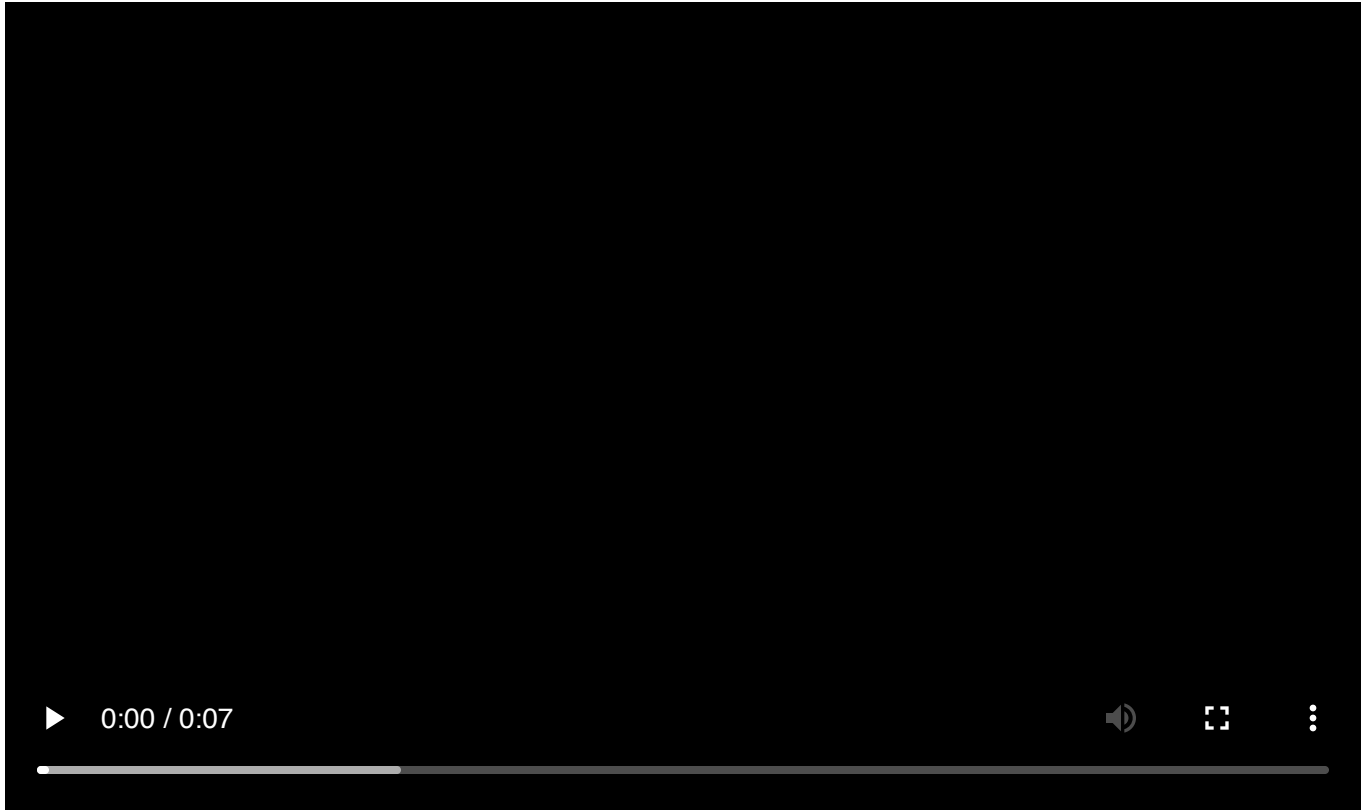
    t.type = floor(random(6)); // neue zufällige form

    t.col = random([ // neue zufällige farbe
        "#e94e2e",

```

```
        "#f2efe6",  
        "#b8b0a5"  
    });  
}  
}
```

rain but numbers and pixels, give it a synthetic and systematic appearance



```
let streams = [];  
let symbolSize = 26;  
let cols;  
  
function setup() {  
    createCanvas(windowWidth, windowHeight);  
    textFont("monospace");  
    textSize(symbolSize);  
    //cols over whole screen  
    cols = floor(width / symbolSize);  
  
    for (let i = 0; i < cols; i++) {  
        // so many as cols are  
        //create steam (Xposition= index*symbolsize)  
        streams.push(new Stream(i * symbolSize));  
    }  
}
```

```

    background(0, 0, 255);
}

function draw() {
  // motion blur with alpha = 35
  //pixel rect
  fill(0, 0, 255, 35);
  noStroke();
  rect(0, 0, width, height);

  //loop that streams move and are draw again after disapar
  for (let s of streams) {
    s.update();
    s.render();
  }
}

class Stream {
  constructor(x) {
    //this für local
    this.x = x;
    this.y = random(-1000, 0);
    this.speed = random(1,5);
    this.length = floor(random(8, 24));
  }

  update() {
    //steam falls down
    this.y += this.speed;

    if (this.y > height + this.length * symbolSize) {
      this.y = random(-500, -100);
      this.speed = random(4, 12);
    }
  }

  render() {
    for (let i = 0; i < this.length; i++) {
      let charY = this.y - i * symbolSize;

      let value = random() > 0.5 ? "0" : "1";

      //transparent
      let alpha = map(i, 0, this.length, 255, 20);

      // Pixel rects

```

```

    let pixelBox = random() > 0.82;

    if (pixelBox) {
        fill(255,20,147);
        rect(this.x - 2, charY - symbolSize + 4, symbolSize, symbolSize);
        fill(255, alpha);
    } else {
        fill(255, alpha);
    }

    text(value, this.x, charY);
}
}
}

function windowResized() {
    resizeCanvas(windowWidth, windowHeight);
}

```

organic forms with typography



```

let chars = "irgend e text";
let spacing = 14; // abstand
let t = 0; // zeitvariable

```

```
function setup() {
  createCanvas(windowWidth, windowHeight);

  textFont('monospace');
  textSize(10);
  textAlign(CENTER, CENTER);

  noStroke();
}

function draw() {

  // farbwechsel
  if (sin(frameCount * 0.01) > 0) {

    background(10, 120, 80);
    fill(255);

  } else {

    background(30, 60, 200);
    fill(255);
  }

  translate(width / 2, height / 2); // ursprung bildschirmmitte

  let index = 0; // zeichenzähler

  // raster aus zeichen
  for (let y = -height; y < height; y += spacing) { // zeilen

    for (let x = -width; x < width; x += spacing) { // spalten

      // wellenbewegung
      let wave =
        sin(x * 0.01 + t) * 120 + // horizontal
        cos(y * 0.01 + t) * 120; // vertikal

      // verzerrung
      let nx = x + sin(y * 0.02 + t) * 40; // x verschoben
      let ny = y + cos(x * 0.02 + t) * 40; // y verschoben

      // kreisförmige struktur
      let d = dist(nx, ny, 0, 0); // abstand zur mitte

      let mask = sin(d * 0.03 - t * 2); // ringförmiges muster
```

```

    if (mask > 0.2) { // nur bestimmte bereiche zeichnen

        let c = chars[index % chars.length]; // nächstes zeichen

        push();

        translate(
            nx + wave * 0.1, // zusätzliche bewegung
            ny
        );

        rotate(
            sin(t + d * 0.01) * 0.2 // drehung
        );

        text(c, 0, 0);

        pop();

        index++;
    }
}

t += 0.03; // zeitvariable erhöhen
}

function windowResized() {
    resizeCanvas(windowWidth, windowHeight);
}

```

"systematic" partical system

▶ 0:00 / 0:07



```
let particles = []; // array für partikel
const num = 800;

function setup() {
  createCanvas(windowWidth, windowHeight);

  stroke(0, 100, 50);
  strokeWeight(1);

  for (let i = 0; i < num; i++) { // 800 partikel erzeugen

    particles.push({

      x: random(width), // random x
      y: random(height), // random y

      angle: random([ // startwinkel nur in 90°-schritten
        0,
        HALF_PI,
        PI,
        PI + HALF_PI
      ])

    });
  }
}
```

```

function draw() {

    background(255, 255, 0, 15); // alpha, spuren sichtbar

    for (let p of particles) { // alle partikel durchlaufen

        let oldX = p.x;
        let oldY = p.y;

        // manchmal richtung ändern
        if (random() < 0.05) {

            p.angle += random([
                -HALF_PI, // 90° links
                HALF_PI    // 90° rechts
            ]);

        }

        // bewegung
        p.x += cos(p.angle) * 8;
        p.y += sin(p.angle) * 8;

        // gepunktete linie
        dottedLine(
            oldX, oldY,
            p.x, p.y,
            1, 3
        );

        // bildschirmrand, auf andere seite springen
        if (p.x < 0) p.x = width;
        if (p.x > width) p.x = 0;

        if (p.y < 0) p.y = height;
        if (p.y > height) p.y = 0;
    }
}

function dottedLine(
    x1, y1,
    x2, y2,
    dotLength,
    gapLength
) {

```

```

let distance = dist(
  x1, y1,
  x2, y2
);

let angle = atan2(
  y2 - y1,
  x2 - x1
);

for (
  let i = 0;
  i < distance;
  i += dotLength + gapLength
) { // in punktabständen vorwärts gehen

  let sx = x1 + cos(angle) * i; //start
  let sy = y1 + sin(angle) * i; //start

  let ex =
    x1 +
    cos(angle) * (i + dotLength); // endpunkt

  let ey =
    y1 +
    sin(angle) * (i + dotLength); // endpunkt

  line(sx, sy, ex, ey); // liniensegment
}
}

```

WEEK3&4 - FRI - MON - STEFANIE AND YANN

presentation

1 snippet : organicFlowField()

perlin noise field - looks organic - also could symbolize this way from countryside to city ->
 fortziehen
 colors to emphasize the organic

▶ 0:00 / 0:07



```
//perlin noise flow field

//array for particles
let particles = [];

//number of particles
const num = 1000;

//noise-scale - how "wild"
const noiseScale = 0.005/2;

function setup() {
  createCanvas(windowWidth, windowHeight);
}

function draw() {
  organicFlowField();
}
```

```

function organicFlowField() {

  //create particles only the first time - setup-ersatz
  if(particles.length === 0) {

    //loop to create vector for each particle
    for(let i = 0; i < num; i ++) {

      //random x and y positions - push inserts vectors in array
      particles.push(
        createVector(random(width), random(height))
      );
    }

    //particle color
    stroke(255,255,0);
  }

  //alpha 0.1 = only slightly overpaints the old frame
  background(0,100,50,0.1);

  //loop to move particles
  for(let i = 0; i < num; i ++) {

    //get particles
    let p = particles[i];

    //paint particles
    point(p.x, p.y);

    //natural random values not like chaotic like random - 3D-noise
    let n = noise(
      p.x * noiseScale,
      p.y * noiseScale,
      frameCount * noiseScale * noiseScale
    );

    //calculate angle (TAU = 2 * PI)
    let a = TAU * n;

    //moves particles - *3 = speed
    p.x += cos(a)*3;
    p.y += sin(a)*3;

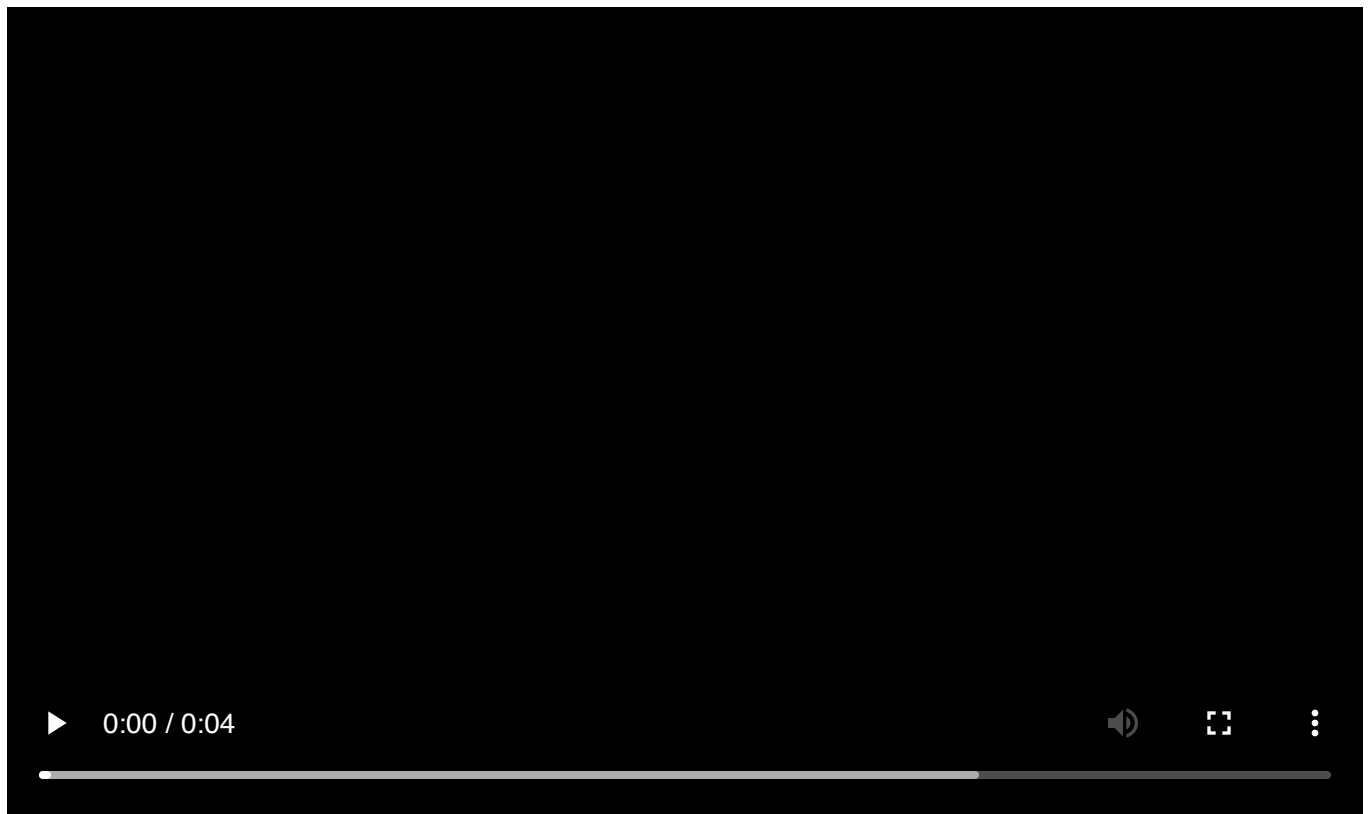
    //if not on screen reset it so there are always num particles
  }
}

```

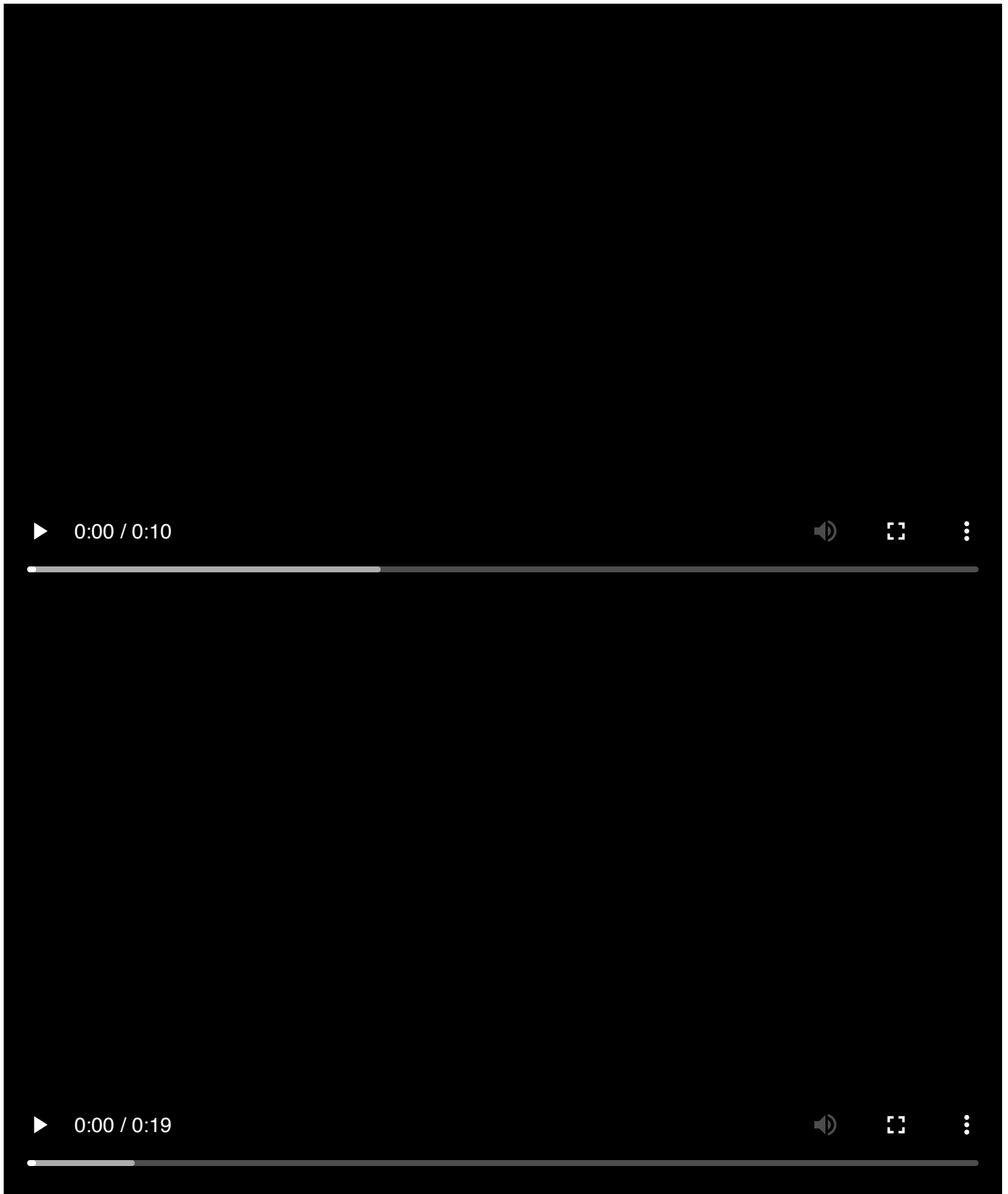
```
    if(  
        p.x < 0 ||  
        p.x > width ||  
        p.y < 0 ||  
        p.y > height  
    ) {  
  
        p.x = 10;  
        p.y = 10;  
    }  
}  
}
```

process

<https://www.youtube.com/watch?v=sZBfLgfsvSk>







2 snippet: snakeText()

organic movements of typography
remembered me of underwater plants



```
function snakeText() {  
  
  //create particles only the first time – setup-ersatz  
  if(snakes.length === 0) {  
    //how many snakes  
    let cols = 20;  
    //loop to create snakes
```

```

for (let i = 0; i < cols; i++) {
  //calculate x coordinate - map() translates values - i=0-19 - 0-width
  let x = map(i, 0, cols - 1, 0, width);

  //push inserts objects in array
  snakes.push({
    x,
    amp: random(80, 200), //wave amplitude
    freq: random(0.004, 0.01), //wave frequency
    phase: random(TWO_PI), //starting angle of sine wave
    speed: random(0.8, 1.8),
    spacing: random(25, 40), //space between words
    size: random(18, 34), //textsize
    offset: random(height) //snakes begin at different heights
  });
}

textFont("Helvetica");
textAlign(CENTER, CENTER);
noStroke();
}

background(202,209,131);
fill(255);

//loop to move snakes
for (let s of snakes) {
  //offset increases with each frame - snakes move downwards
  s.offset += s.speed;
  //every snake consists of 80 words
  for (let i = 0; i < 80; i++) {
    //calculate y coordinate (evenly spacing- moves downwards)
    //if out of screen reappears at the top
    let y = (i * s.spacing + s.offset) % (height + 200);
    //snake shape is formed here - wave movement , wave form, moves wave ,
    increases movement
    let wave = sin(y * s.freq + s.phase) * s.amp;
    //let text do this wave movements
    let x = s.x + wave;
    let word = txt;

    push();
    translate(x, y);
    //rotates text
    rotate(sin(y * s.freq) * 0.2);
    textSize(s.size);
  }
}

```

```
    text(word, 0, 0);  
    pop();  
  }  
}  
}
```

3 snippet: snakeLetters()

▶ 0:00 / 0:05



```
function snakeLetters() {  
  
  //create particles only the first time - setup-ersatz  
  if(snakes.length === 0) {  
    //how many snakes  
    let cols = 20;  
    //loop to create snakes  
    for (let i = 0; i < cols; i++) {  
      //calculate x coordinate - map() translates values - i=0-19 - 0-width  
      let x = map(i, 0, cols - 1, 0, width);  
  
      //push inserts objects in array  
      snakes.push({  
        x,  
        amp: random(80, 200), //wave amplitude  
        freq: random(0.004, 0.01), //wave frequency  
        phase: random(TWO_PI), //starting angle of sine wave  
      });  
    }  
  }  
}
```

```

    speed: random(0.8, 1.8),
    spacing: random(25, 40), //space between words
    size: random(18, 34), //textsize
    offset: random(height) //snakes begin at different heights
  });
}

textFont("Helvetica");
textAlign(CENTER, CENTER);
noStroke();
}

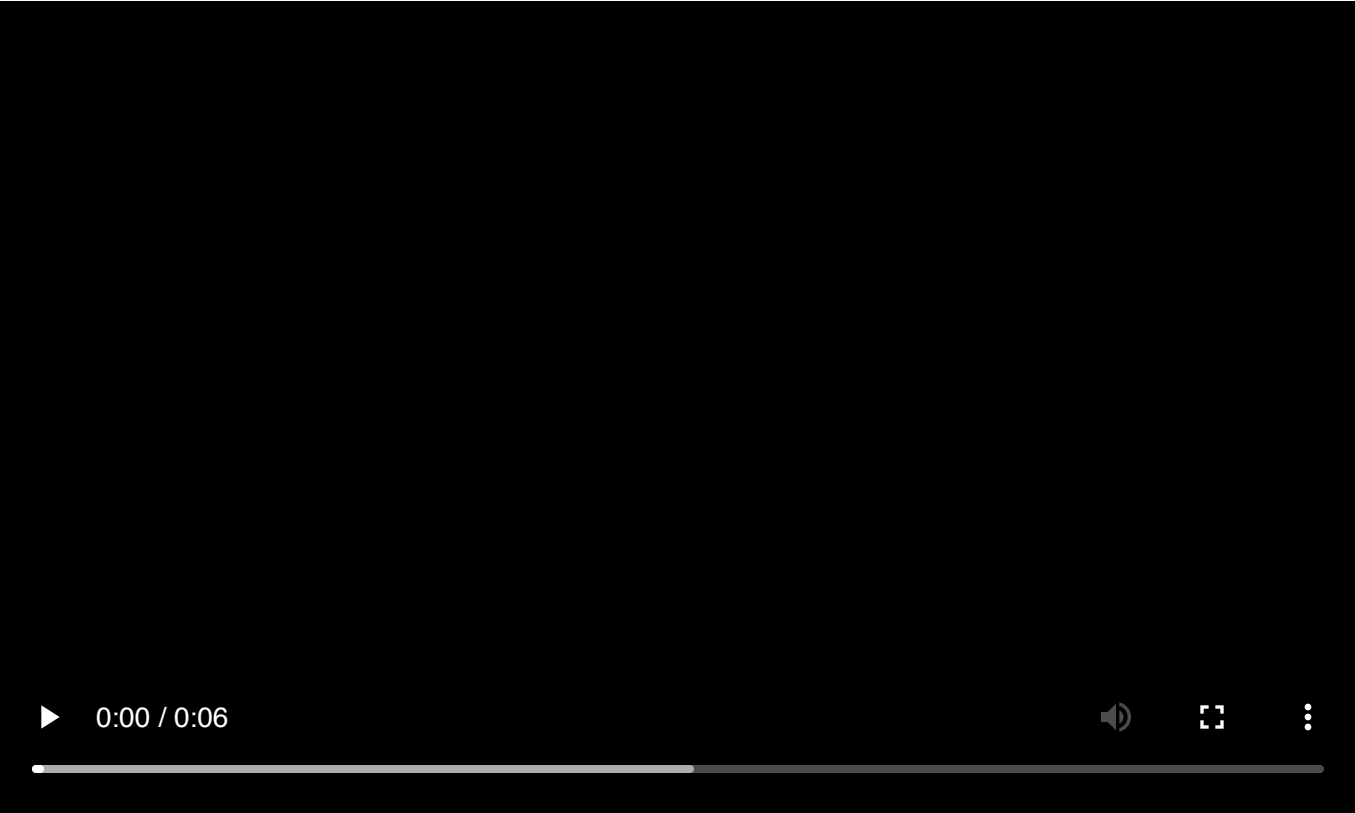
background(202,209,131);
fill(102,2,60);

//loop to move snakes
for (let s of snakes) {
  //offset increases with each frame – snakes move downwards
  s.offset += s.speed;
  //every snake consists of 80 words
  for (let i = 0; i < 80; i++) {
    //calculate y coordinate (evenly spacing– moves downwards)
    //if out of screen reappears at the top
    let y = (i * s.spacing + s.offset) % (height + 200);
    //snake shape is formed here – wave movement , wave form, moves wave ,
    increases movement
    let wave = sin(y * s.freq + s.phase) * s.amp;
    //let text do this wave movements
    let x = s.x + wave;
    //this always takes only letter from the string
    let char = txt[i % txt.length];

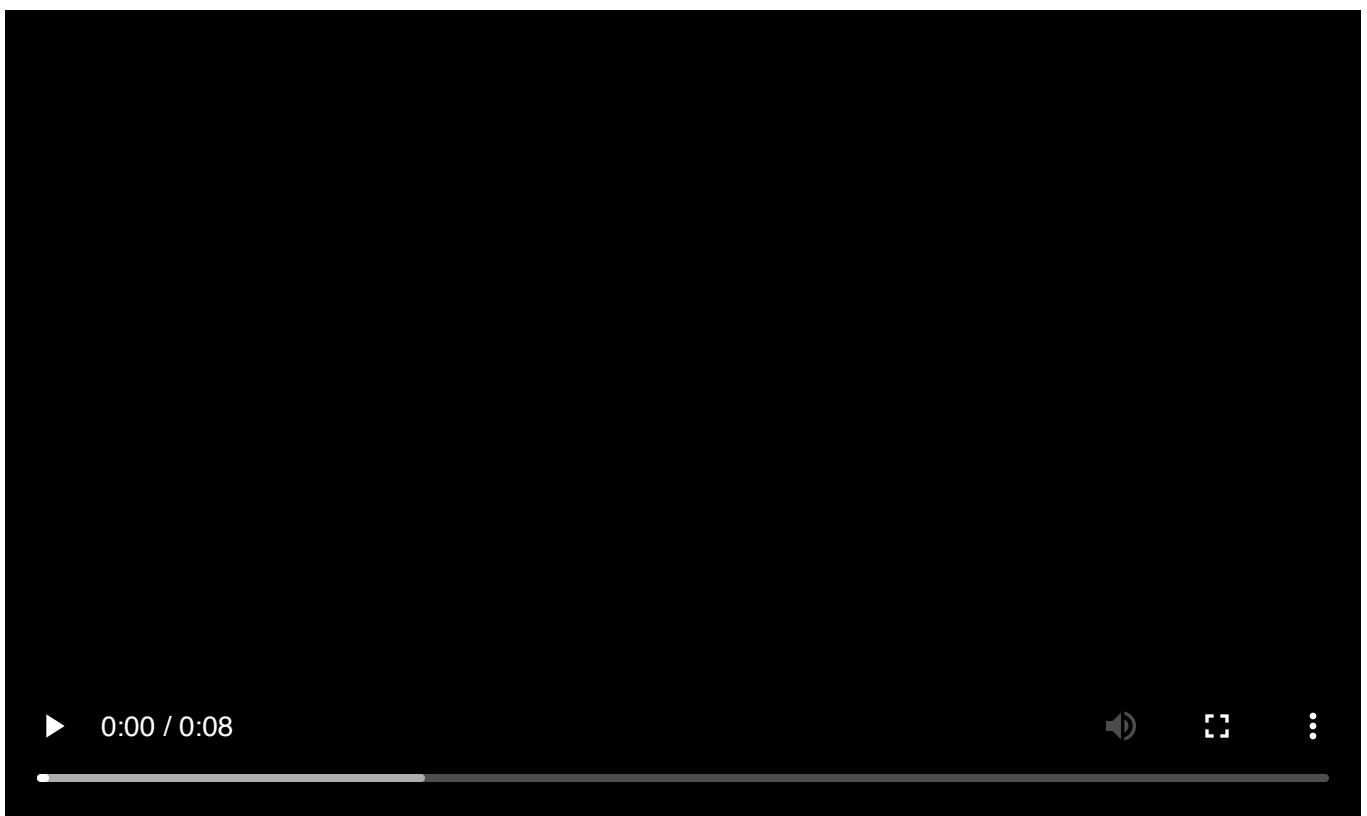
    push();
    translate(x, y);
    //rotates text
    rotate(sin(y * s.freq) * 0.2);
    textSize(s.size);
    text(char, 0, 0);
    pop();
  }
}
}

```

process







4 snippet: bothDynamics()

a grid which symbolizes the synthetic, systematic and pulsating random green circles to symbolize the organic

▶ 0:00 / 0:06



```
function bothDynamics() {  
  
  //create particles only the first time – setup-ersatz  
  if(cell == null) {  
    // rectangle drawn from the center  
    rectMode(CENTER);  
    noFill();  
    stroke(255);  
  }  
  
  background(150);  
  //calculates width of a cell  
  cell = width / grid;  
  
  //goes through each column  
  for (let x = 0; x < grid; x++) {  
    //goes through each cell  
    for (let y = 0; y < grid * 1.7 ; y++) {  
      //calculates x position of center of each cell  
      let px = x * cell + cell / 2;  
      //calculates y position of center of each cell  
      let py = y * cell + cell / 2;  
  
      push();  
      //moves origin to current grid cell  
      translate(px, py);
```



```

        stroke(255);
        strokeWeight(1);
        //draws cell but little bit smaller
        rect(0, 0, cell * 0.95, cell * 0.95);
        //circle function
        drawRingPack(x, y);
        pop();
    }
}
}

//circle function gx,gy coordinates of cells
function drawRingPack(gx, gy) {
    // animation
    let t = frameCount * 0.003;
    // number of rings
    let rings = floor(random(3, 10));
    // size of rings
    let maxSize = random(cell * 0.2, cell * 0.9);
    // pulsation
    maxSize += sin(t + gx * 0.3 + gy * 0.2) * 0.0003;
    // slight offset
    let offsetX = random(-8, 8);
    let offsetY = random(-8, 8);

    //for loop for all circles
    for (let i = 0; i < rings; i++) {
        //every next circle becomes smaller
        let size = maxSize - i * random(5, 12);
        //if circle size becomes negative, skip loop
        if (size <= 0) continue;
        // glow effect
        drawingContext.shadowBlur = 12;
        //color of glow
        drawingContext.shadowColor = "white";
        // random stroke
        strokeWeight(random(0.5, 3));
        // alpha random = random transparency
        stroke(0, random(255), 0, random(120, 255));
        //draws ellipse
        ellipse(
            offsetX,
            offsetY,
            size
        );
    }
}

```

```

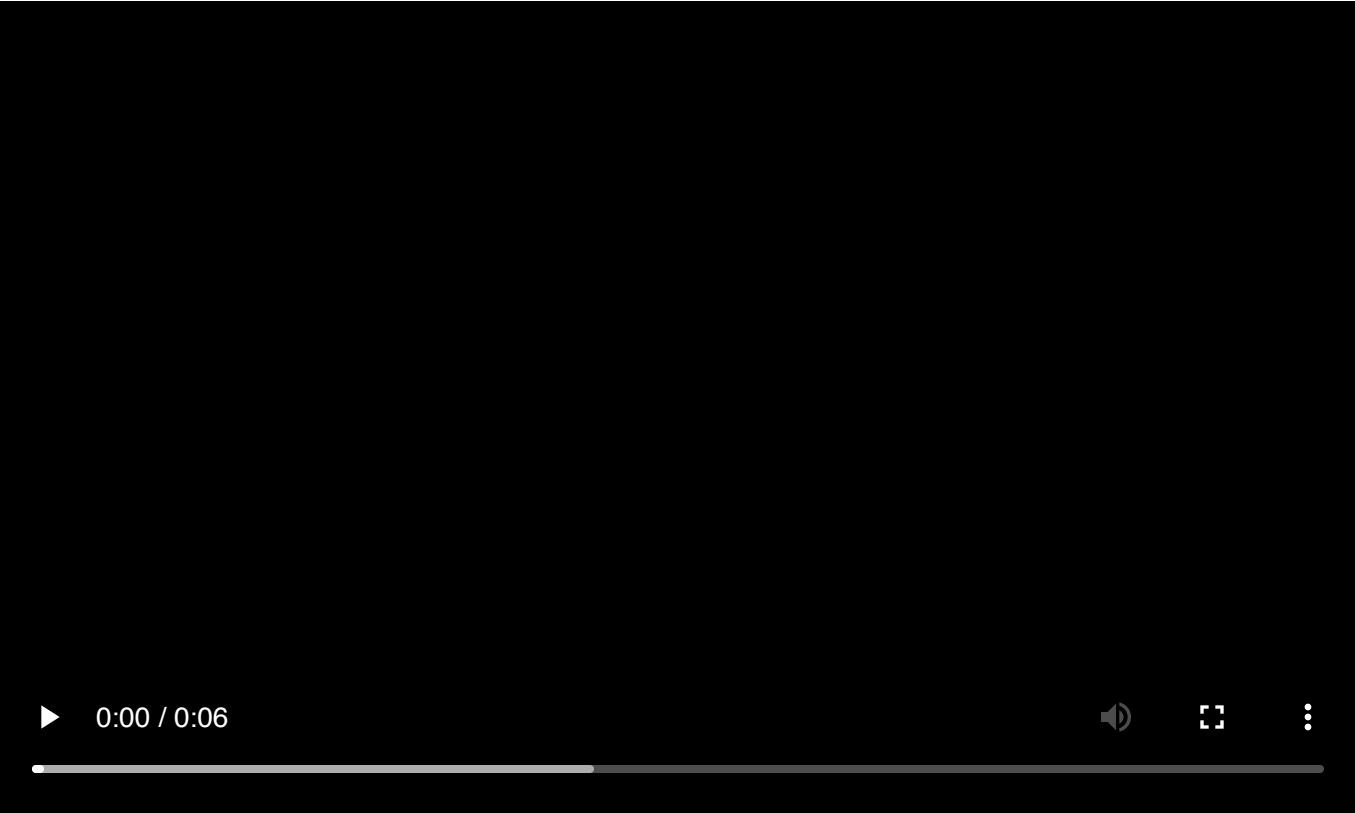
// sometimes second ringpackage over it
if (random() > 0.6) {
  //number and size of circles in second ringpackage
  let rings2 = floor(random(2, 6));
  let size2 = maxSize * random(0.3, 0.8);
  //offset vor second ringpackage
  let ox2 = offsetX + random(-10, 10);
  let oy2 = offsetY + random(-10, 10);

  //loop for second ringpackage
  for (let i = 0; i < rings2; i++) {
    //circles become smaller
    let s = size2 - i * random(4, 10);
    //if smaller than 0 skip loop
    if (s <= 0) continue;
    strokeWeight(random(0.5, 2.5));
    stroke(255, random(80, 220));
    //circle of second ringpackage
    ellipse(ox2, oy2, s);
  }
}
}

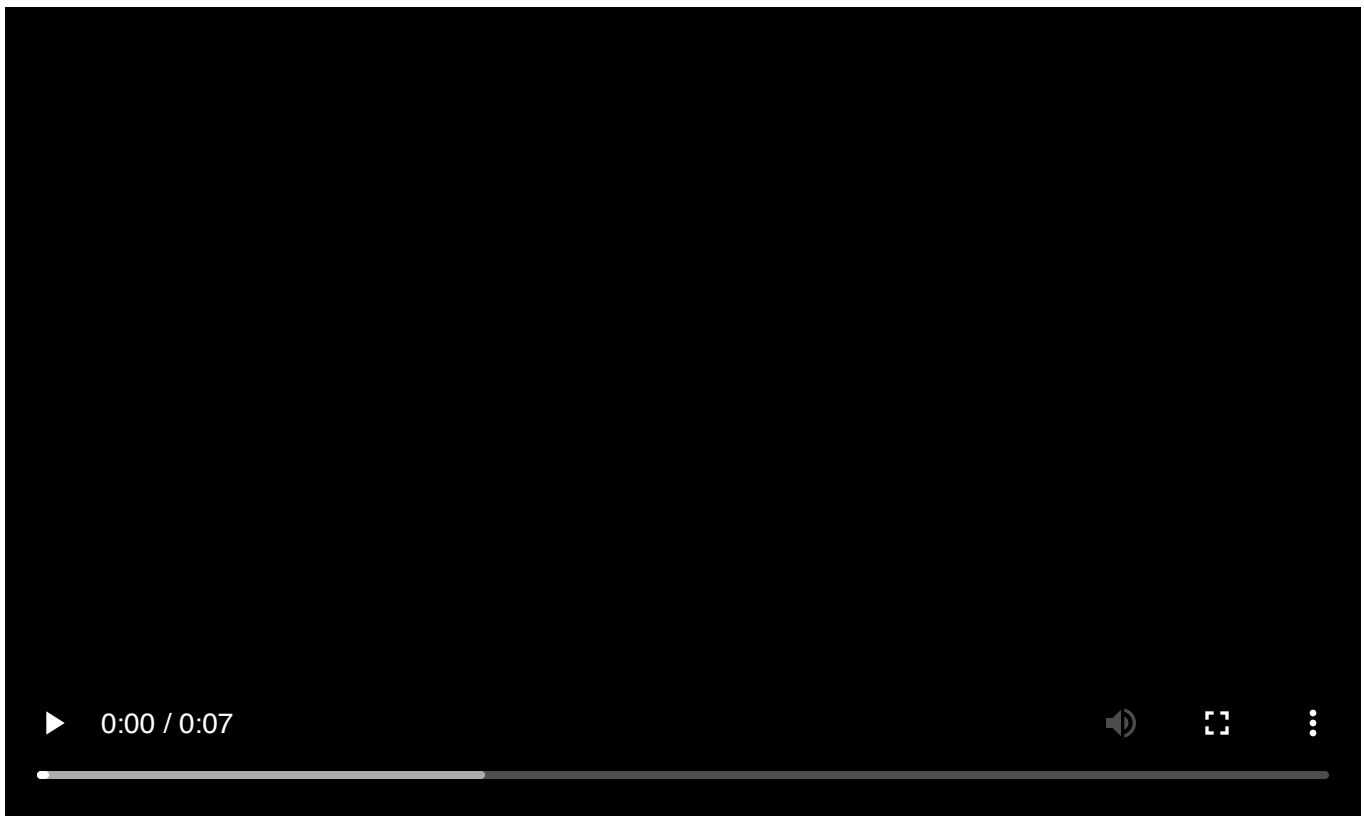
```

process









coaching

perlin noise was created to achieve this kind of organic movement

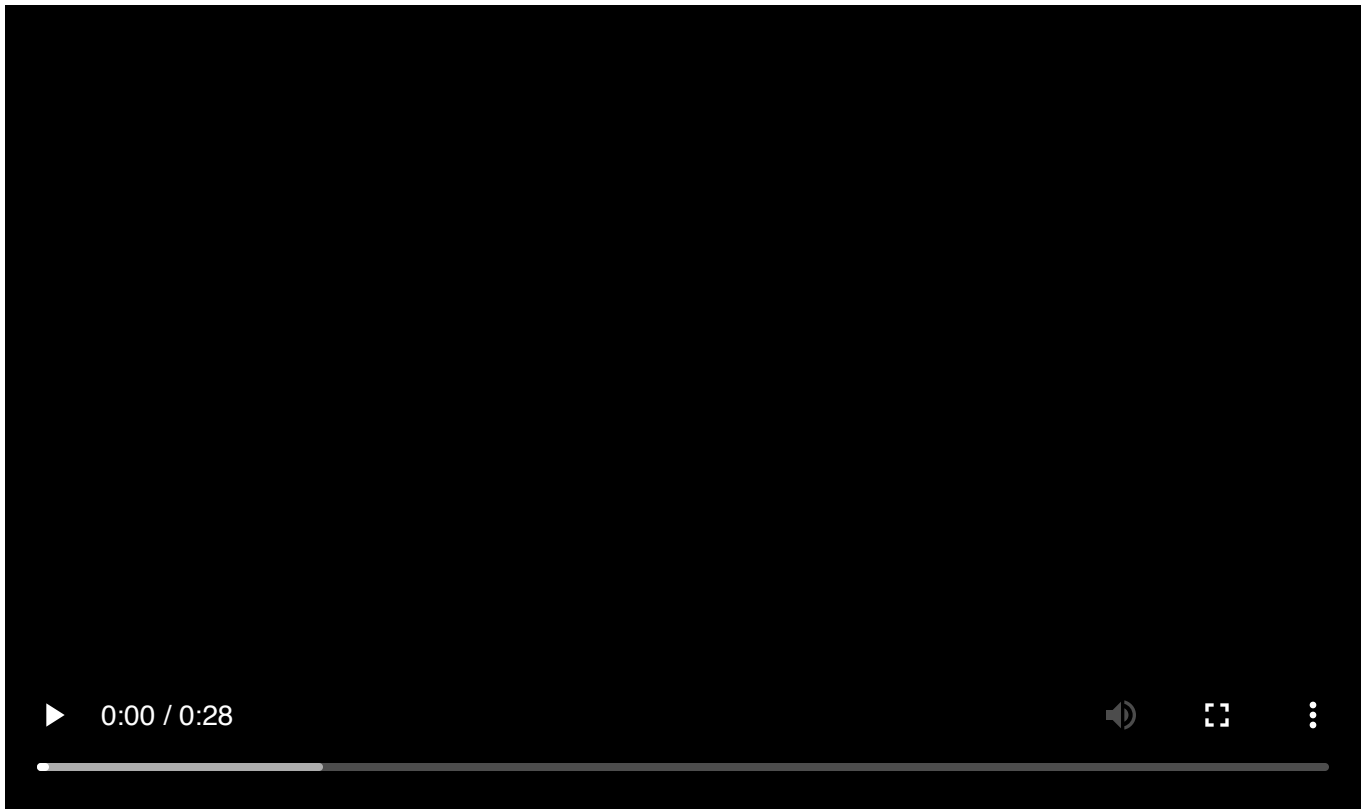
https://en.wikipedia.org/wiki/Perlin_noise

//moves particles - 3 = speed

*p.x += round(cos(a)*3);*

*p.y += round(sin(a)*3);*

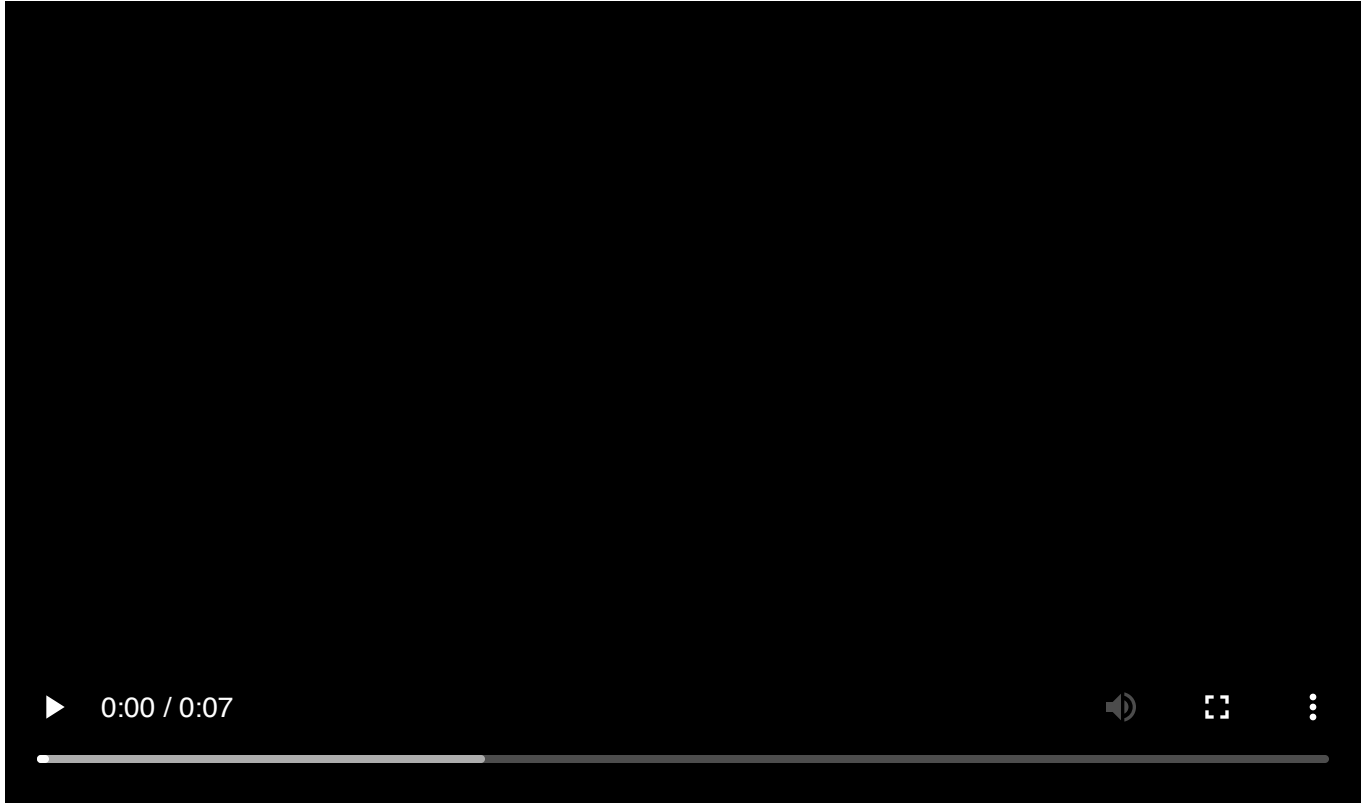
rounded numbers = stricter, less smooth movements -> maybe a way to get a transition from one visual to another by slowly changing the code to a more synthetic



find way to combine organic and synthetic in one

WEEK5

typoflower rotating - word "organic"



▶ 0:00 / 0:10



```
let chars = "organic";
let t = 0; // zeitvariable

function setup() {
  createCanvas(windowWidth, windowHeight);
}

function draw() {
  flower();
}

function flower() {
  background(255);

  textFont("Sofia Pro");
  textAlign(CENTER, CENTER);

  translate(width / 2, height / 2 + 40); // ursprung in mitte der blume

  t += 0.01; // zeit erhöhen

  let index = 0; // zähler für buchstabenauswahl

  // Blütenblätter
  for (let a = 0; a < TWO_PI; a += 0.08) { // kreis durchlaufen
```

```

let r =
  180 * sin(6 * a) + // 6 blütenblätter
  20 * sin(frameCount * 0.02 + a * 3);

let x = cos(a + t * 0.3) * r;
let y = sin(a + t * 0.3) * r;

let c = chars[index % chars.length]; // nächster buchstabe aus wort

push();

translate(x, y);
rotate(a);

fill(0);

let s = map(abs(r), 0, 180, 10, 26); // grösse abhängig vom radius
textSize(s);

text(c, 0, 0);

pop();

index++; // nächster buchstaben
}

// blütenmitte
for (let i = 0; i < 120; i++) { // 120 zeichen

  let angle = random(TWO_PI); // zufälliger winkel
  let radius = random(10, 50); // zufälliger abstand vom zentrum

  let x = cos(angle) * radius; // x-position
  let y = sin(angle) * radius; // y-position

  let c = chars[(i + frameCount) % chars.length]; // animierter
buchstabenwechsel

  fill(255, 255, 0);

  textSize(20);

  text(c, x, y);
}

// Stiel

```



```

for (let y = 80; y < 300; y += 14) { // punkte

    let x = sin(y * 0.03 + t * 2) * 10; // wellenbewegung

    fill(0, 100, 50);

    textSize(16);

    text("|", x, y);
}

// blatt links
for (let a = -1.5; a < 1.5; a += 0.15) { // blattkurve

    let r = 60 * cos(a); // radiusform

    let x = -30 + cos(a + 2.2) * r; // x-position
    let y = 170 + sin(a + 2.2) * r; // y-position

    fill(0, 100, 50);

    textSize(14);

    text("aaa", x, y);
}

// Blatt rechts
for (let a = -1.5; a < 1.5; a += 0.15) {

    let r = 60 * cos(a);

    let x = 70 + cos(a - 2.2) * r;
    let y = 210 + sin(a - 2.2) * r;

    fill(0, 100, 50);

    textSize(14);

    text("aaa", x, y);
}
}

```

pulsating flowers -> idea to do them audio-reactive so it pulsate to sound

▶ 0:00 / 0:09



```
let chars = "organic";
let t = 0;

function setup() {
  createCanvas(windowWidth, windowHeight);
}

function draw() {
  background(255)
  flower(250,200,120,40,700,100);
  flower(550,500,160,60,400,100);
  flower(900,150,80,40,750,100);
  flower(1200,400,140,40,750,100);
}

function flower(x, y, blütenradius, mittelradius, stielhöhe, blattbreite) {

  push();
  translate(x, y);

  t += 0.01;
  let index = 0;

  //pulsierend statt drehend
  for (let a = 0; a < TWO_PI; a += 0.08) {
```

```

let pulse = sin(frameCount * 0.03 + a * 6); //wellenbewegung puls

let r =
  blütenradius * sin(6 * a) + //blütengrösse
  blütenradius * 0.15 * pulse; // puls grösser kleiner

let x1 = cos(a) * r;
let y1 = sin(a) * r;

let c = chars[index % chars.length];

push();
translate(x1, y1);
fill(0);
textSize(18);
text(c, 0, 0);
pop();

index++;
}

// Blütenmitte
for (let i = 0; i < 120; i++) {

  let angle = random(TWO_PI);
  let radius = random(0, mittelradius);

  let x1 = cos(angle) * radius;
  let y1 = sin(angle) * radius;

  let c = chars[(i + frameCount) % chars.length];

  fill(255, 255, 0);
  textSize(20);
  text(c, x1, y1);
}

// Stiel
for (let yStem = 0; yStem < stielhöhe; yStem += 14) {
  let xStem = sin(yStem * 0.03) * 10;

  fill(0, 100, 50);
  textSize(16);
  text("|", xStem, yStem);
}

```

```
// Blatt links
for (let a = -1.5; a < 1.5; a += 0.15) {
  let r = blattbreite * cos(a);

  fill(0, 100, 50);
  textSize(14);

  text(
    "aaa",
    -30 + cos(a + 2.2) * r,
    170 + sin(a + 2.2) * r
  );
}

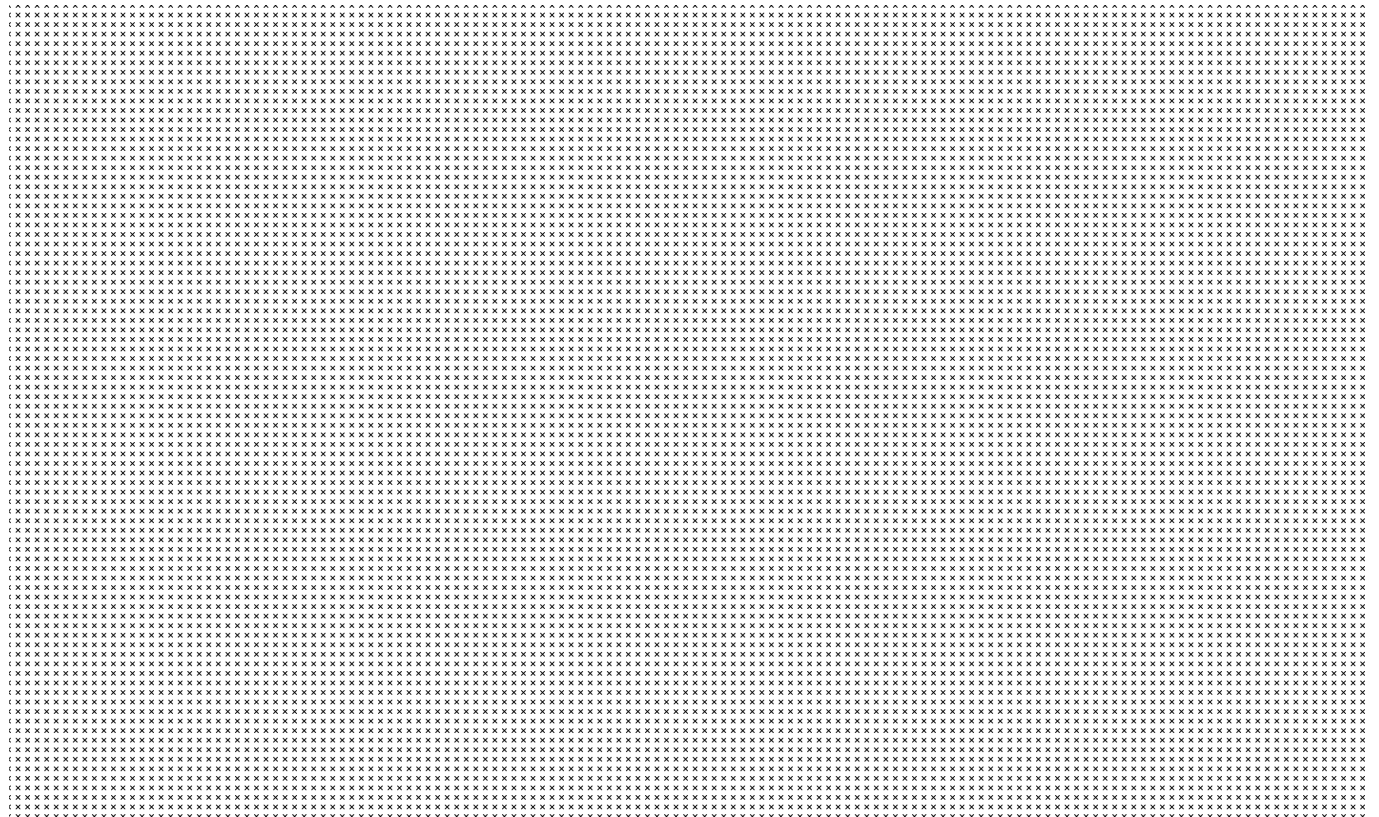
// Blatt rechts
for (let a = -1.5; a < 1.5; a += 0.15) {
  let r = blattbreite * cos(a);

  fill(0, 100, 50);
  textSize(14);

  text(
    "aaa",
    70 + cos(a - 2.2) * r,
    210 + sin(a - 2.2) * r
  );
}

pop();
}
```

pixelblume



```
function setup() {  
  createCanvas(windowWidth, windowHeight);  
}  
  
function draw() {  
  pixelRaster();  
}  
  
function pixelRaster() {  
  background(255);  
  
  let s = 10;  
  
  for (let x = 0; x < width; x += s) {  
    for (let y = 0; y < height; y += s) {  
      stroke(0);  
  
      line(x-2, y-2, x+2, y+2);  
      line(x-2, y+2, x+2, y-2);  
    }  
  }  
}
```

▶ 0:00 / 0:05



```
function setup() {  
  createCanvas(windowWidth, windowHeight);  
}  
  
function draw() {  
  pixelRaster();  
}  
  
function pixelRaster() {  
  background(255);  
  
  let s = 10; // abstand/grösse des rasters  
  
  for (let x = 0; x < width; x += s) { // durchläuft alle spalten im grid  
  
    let d = random(-20, 20); // zufällige horizontale verschiebung für diese  
    spalte  
  
    for (let y = 0; y < height; y += s) { // durchläuft alle zeilen im grid  
  
      d += random(-0.4, 0.4); // zufällige änderung der verschiebung  
  
      stroke(  
        noise(x * .01, y * .002) > .5 ? 0 : 255  
      ); // perlin-noise black or white
```

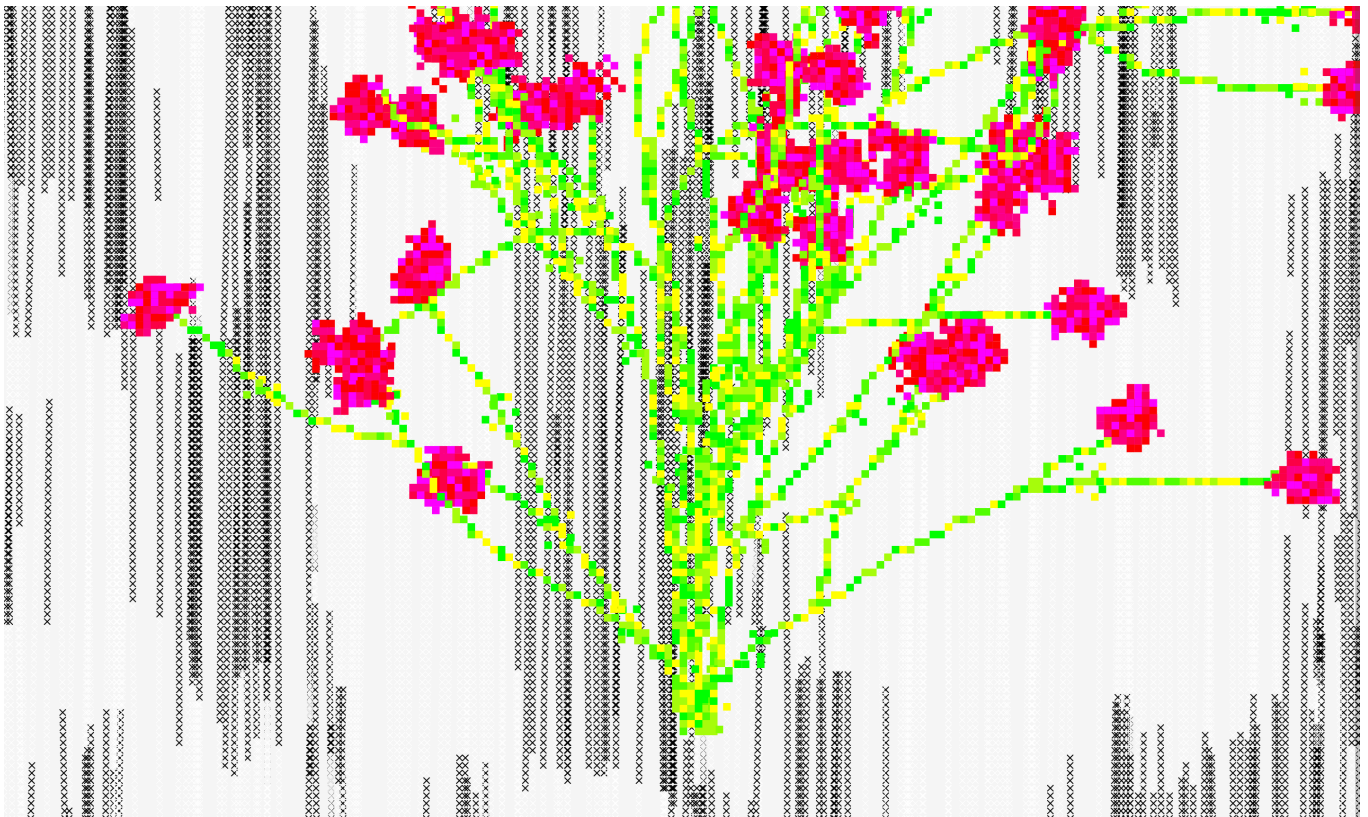
```

    line(
      x + d - 2, y - 2,
      x + d + 2, y + 2
    ); // erste diagonale X

    line(
      x + d - 2, y + 2,
      x + d + 2, y - 2
    ); // zweite diagonale X

  }
}
}

```



```

let S = 8;

function setup() {
  createCanvas(windowWidth, windowHeight);
  background(245);

  drawBackground();

  let rootX = width * 0.5;
  let rootY = height - 100;

```

```

for (let i = 0; i < 5; i++) {
  growBranch(
    rootX + random(-20, 20),
    rootY,
    -PI / 2 + random(-0.15, 0.15),
    random(80, 120)
  );
}
}

function drawBackground() {

  strokeWeight(1);

  for (let x = 0; x < width; x += S) {

    let drift = random(-30, 30);
    let phase = random(1000);

    for (let y = 0; y < height; y += S) {

      drift += map(
        noise(phase, y * 0.003),
        0, 1,
        -0.4, 0.4
      );

      let xx = x + drift;

      let n = noise(x * 0.01, y * 0.002);

      if (n > 0.52) {
        stroke(0);
      } else {
        stroke(255);
      }

      drawCross(xx, y, 3);
    }
  }
}

function drawCross(x, y, r) {
  line(x - r, y - r, x + r, y + r);
  line(x - r, y + r, x + r, y - r);
}

```



```

}

function growBranch(x, y, angle, steps) {

  let cx = x;
  let cy = y;

  for (let i = 0; i < steps; i++) {

    fill(random([
      "#00ff00",
      "#55ff00",
      "#aaff00",
      "#ffff00"
    ]));

    noStroke();
    rect(
      round(cx / S) * S,
      round(cy / S) * S,
      S,
      S
    );

    if (random() < 0.02) {
      leaf(cx, cy);
    }

    cx += cos(angle) * S;
    cy += sin(angle) * S;

    angle += random(-0.12, 0.12);

    if (random() < 0.02 && steps > 40) {

      growBranch(
        cx,
        cy,
        angle + random(-0.8, 0.8),
        steps * 0.6
      );
    }
  }

  flowerCluster(cx, cy);
}

```

```

function leaf(x, y) {

  for (let i = 0; i < 6; i++) {

    fill(random([
      "#00ff00",
      "#88ff00",
      "#ffff00"
    ]));

    rect(
      x + floor(random(-2, 3)) * S,
      y + floor(random(-2, 3)) * S,
      S,
      S
    );
  }
}

function flowerCluster(cx, cy) {

  let cells = [{x:0, y:0}];

  for (let i = 0; i < 300; i++) {

    let p = random(cells);

    cells.push({
      x: p.x + floor(random(-1, 2)),
      y: p.y + floor(random(-1, 2))
    });
  }

  for (let p of cells) {

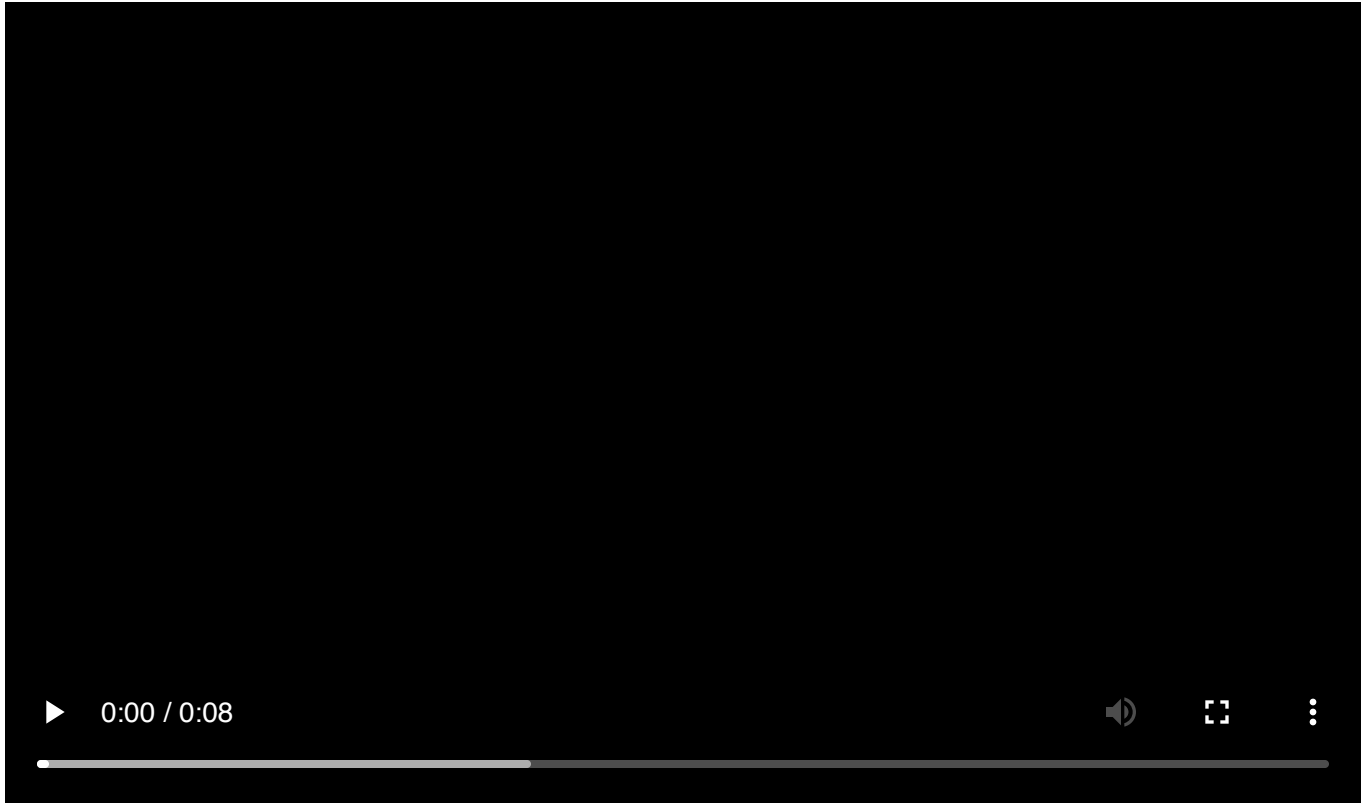
    fill(random([
      "#ff0000",
      "#ff0044",
      "#ff0088",
      "#ff00ff"
    ]));

    rect(
      cx + p.x * S,
      cy + p.y * S,

```

```
    S,  
    S  
  );  
}  
}
```

random generated flowers



```
function setup() {  
  
  createCanvas(windowWidth, windowHeight)  
  //frameRate(1.5)  
}  
  
function draw() {  
  
  if(frameCount%20===0){  
    pixelFlower()  
  }  
}  
  
function pixelFlower(){  
background(255)  
let s=10; //cell-size  
  
for(let x=0;x<width;x+=s){ // über breite des canvas
```

```

let d=random(-20,20);      // horizontaler versatz pro spalte

for(let y=0;y<height;y+=s){  // über höhe des canvas

d+=random(-.4,.4);          // zufällige drift "organische Bewegung"

stroke(noise(x*.01,y*.002)>.5?0:255); // farbe abhängig von noise

line(x+d-2,y-2,x+d+2,y+2);   // /
line(x+d-2,y+2,x+d+2,y-2);   // \
}}

for(let k=0;k<6;k++)          // anzahl äste
ast(                           // ast function
width/2+random(-20,20),       // start x mitte mit random
height,                       // start y
-PI/2+random(-.2,.2),         // richtung nach oben random
90                             // länge
);

function ast(x,y,a,n){        // ast function

for(let i=0;i<n;i++){         // läuft n schritte entlang eines astes

fill(random(["#006432", "#a2c72b","#173b19"]));
noStroke();

rect(                           // raster rechteck
round(x/s)*s,                  // x wird ins raster eingerastet
round(y/s)*s,                  // y wird ins raster eingerastet
s,s                             // rect gröesse
);

if(random()<.02)                // kleine wahrscheinlichkeit für
verzweigung                    // neuer zweig
ast(                            // gleiche position
x,
y,
a+random(-.8,.8),              // veränderter winkel
n*.6                           // veränderte länge für neuen ast
);

x+=cos(a)*s;                   // bewegt position in richtung winkel a
(x)
y+=sin(a)*s;                   // bewegt position in richtung winkel a
(y)

```

```

a+=random(-.12,.12);          // random richtungsänderung
}

for(let i=0;i<500;i++){      // blüte

fill(random([ "#ffff00","#f06","#f0f"]));

rect(
round((x+randomGaussian()*35)/s)*s, // random streuung um x
round((y+randomGaussian()*35)/s)*s, // random streuung um y
s,                                // grösse
s
);
}
}
}

```

coaching

instead of changing complex code - put hydra effect on top to play with the visual

Yann also explained to me that I can put a fix frameCount in the draw-cicle so all the snippets have a similar speed

```

function draw() {
  if(frameCount%2===0){
    functionName()
  }
}

```

▶ 0:00 / 0:06

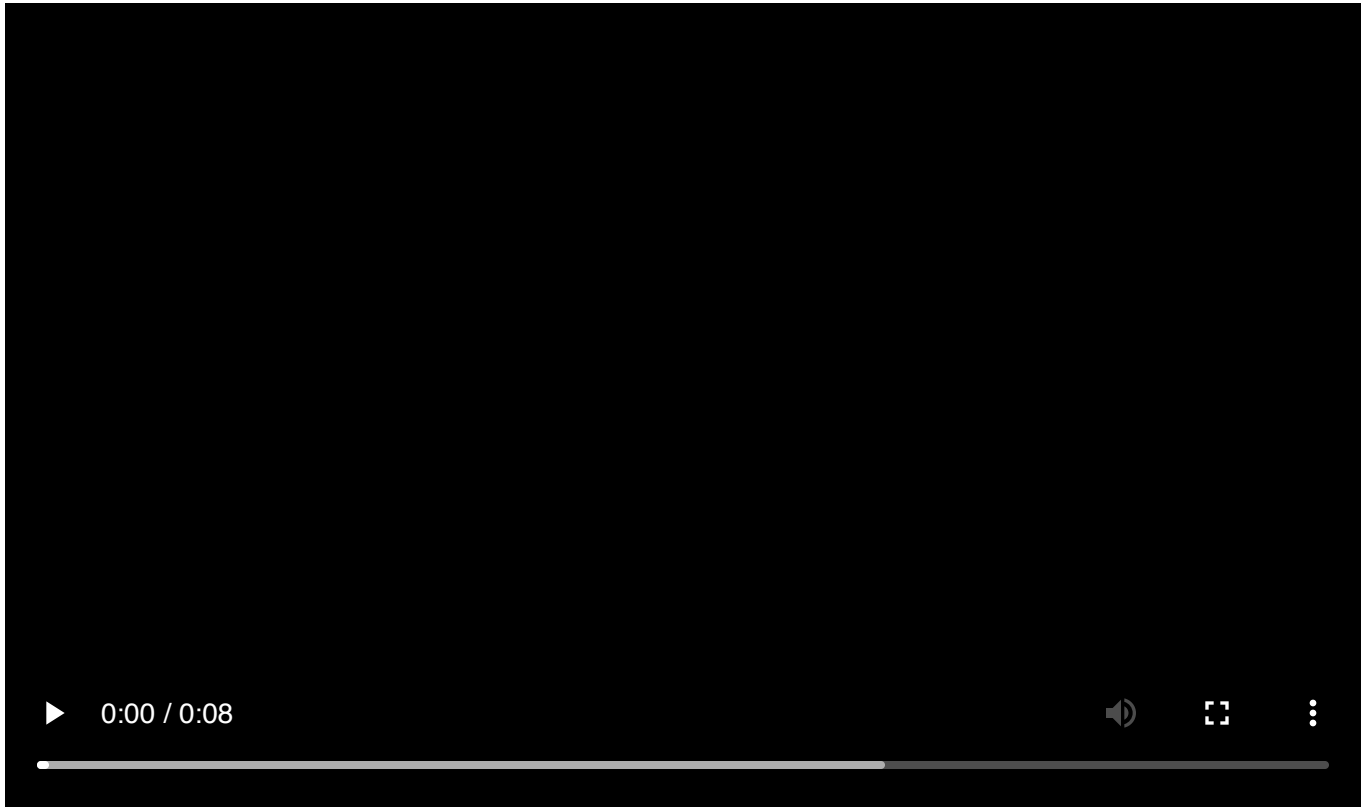


```
src(s0)  
  .repeat(10,10)  
  .out()
```

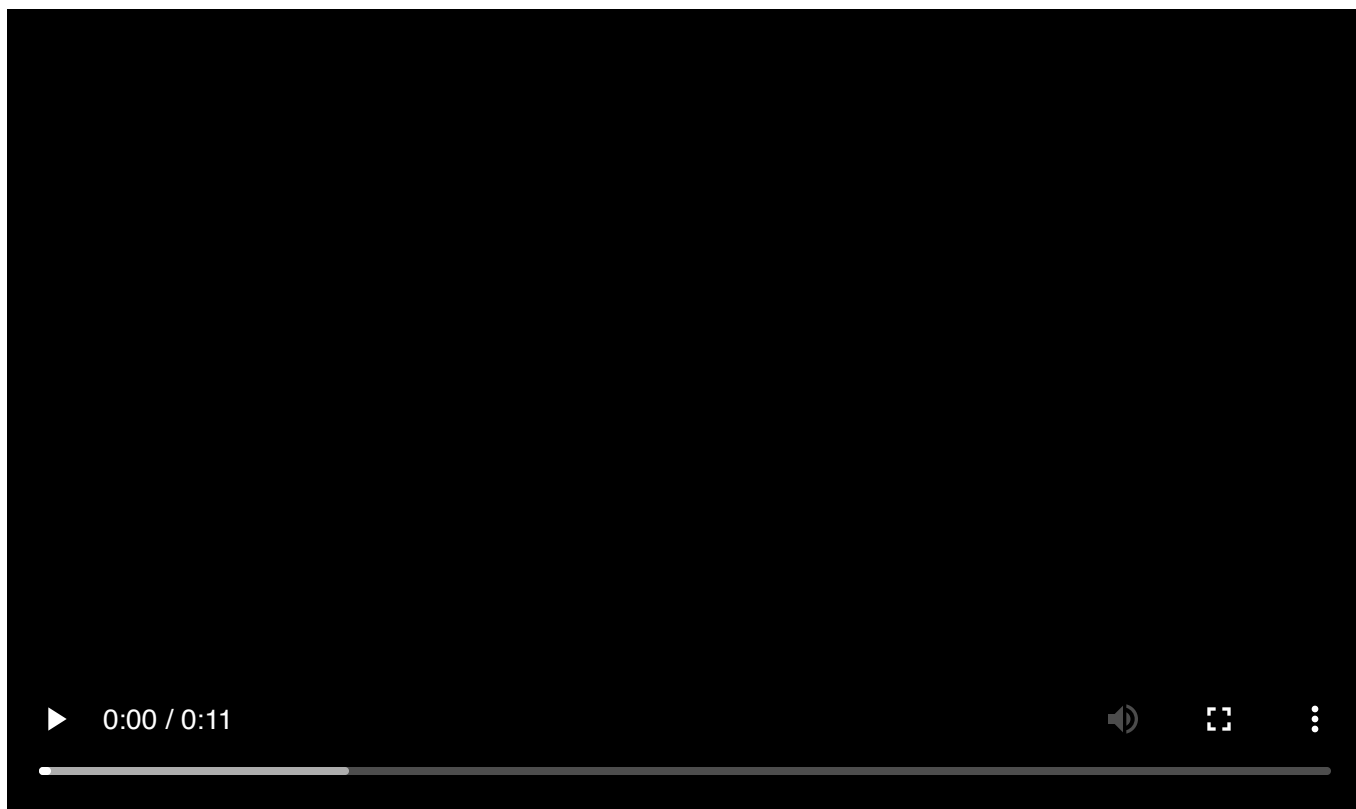
▶ 0:00 / 0:07



```
src(s0)
  .pixelate(60,60)
  .repeat(6,6)
  .rotate(()=>time*0.2)
  .out()
```



```
src(s0)
  .kaleid(8)
  .pixelate(40,40)
  .out()
```



```
src(s0)
  .repeat(2,2)
  .kaleid(2)
  .out()
```

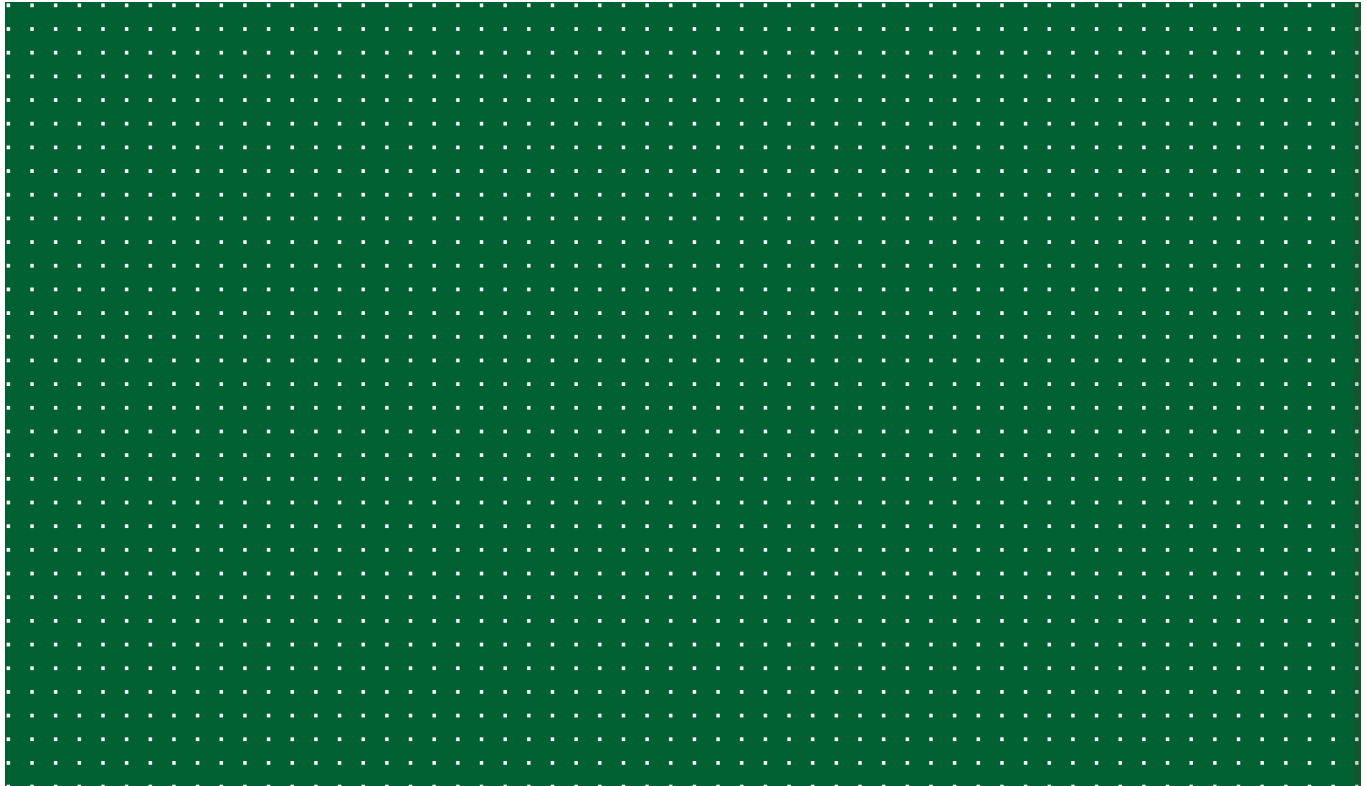



```

src(s0)
  .blend(src(o0),0.5)
  .modulate(
    noise(3,0.02),
    0.03
  )
  .scrollX(0.00005)
  .out()

```

a system - grid for the synthetic part



```

let s = 25;
let c, r, g = [];

const pal = [
  "#006432",
  "#a2c72b",
  "#173b19",
  "#ffff00",
  "#ff0066",
  "#ff00ff"
];

function setup() {
  createCanvas(windowWidth, windowHeight);

```

```
    textAlign(CENTER, CENTER);
    textSize(s * 0.75);
    textFont("monospace");

    initGrid();
}

function draw() {
    background(0);

    drawGrid();
}

function initGrid() {
    c = floor(width / s);
    r = floor(height / s);

    g = new Array(r);

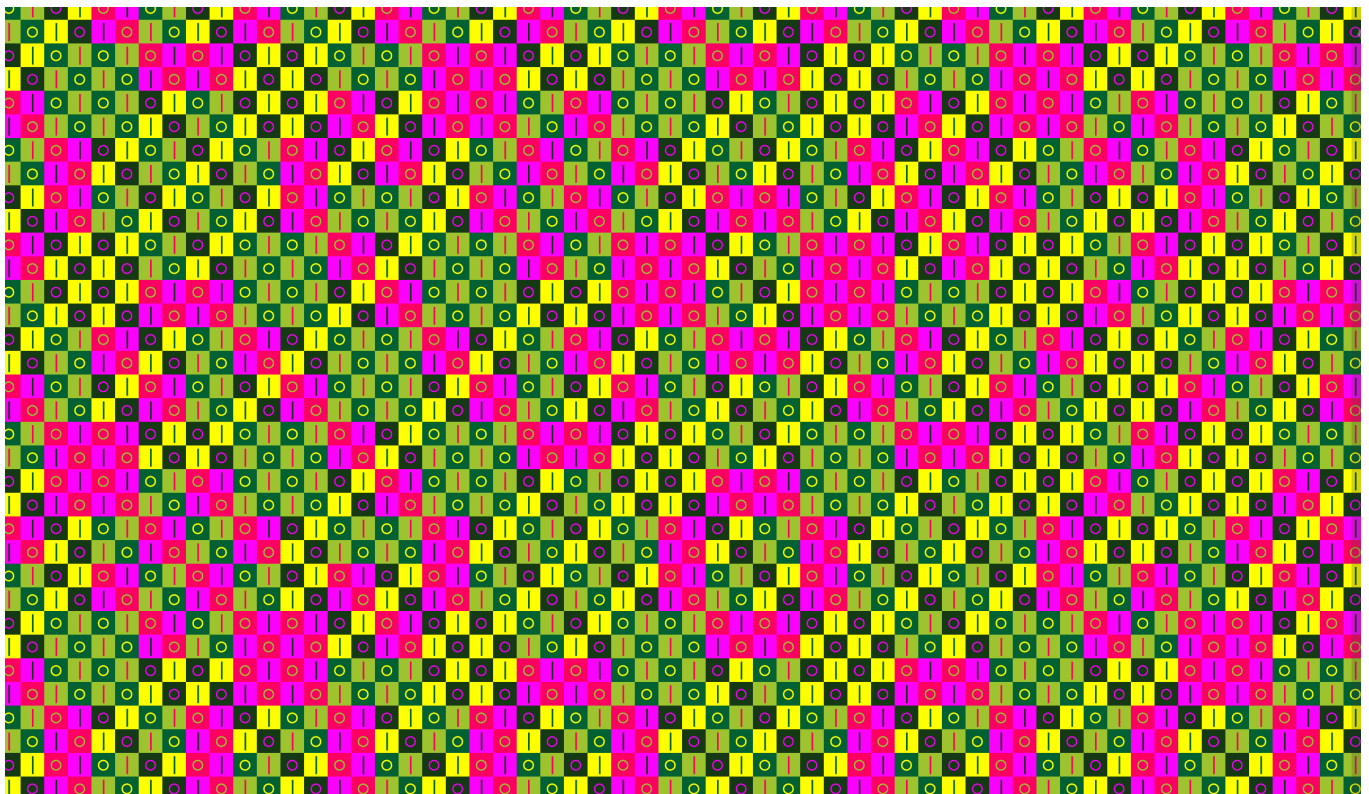
    for (let y = 0; y < r; y++) {
        g[y] = new Array(c).fill(0);
    }
}

function drawGrid() {
    for (let y = 0; y < r; y++) {
        for (let x = 0; x < c; x++) {

            let v = g[y][x];

            fill(pal[v]);
            noStroke();
            rect(x * s, y * s, s, s);

            fill(255);
            text(".", x * s + s / 2, y * s + s / 2);
        }
    }
}
```



```
let s = 25;
let c, r, g = [];

const pal = ["#006432", "#a2c72b", "#173b19", "#ffff00", "#ff0066",
"#ff00ff"];

function setup() {
  createCanvas(windowWidth, windowHeight);
  textAlign(CENTER, CENTER);
  textSize(s * 0.75);
  textFont("monospace");

  initGrid();
}

function draw() {
  background(0);
  drawGrid();
}

function initGrid() {
  c = floor(width / s);
  r = floor(height / s);
  g = [];

  for (let y = 0; y < r; y++) {
```

```

    g[y] = [];
    for (let x = 0; x < c; x++) {
        g[y][x] = (x ^ y) % 6;
    }
}

function drawGrid() {
    for (let y = 0; y < r; y++) {
        for (let x = 0; x < c; x++) {
            let v = g[y][x];
            fill(pal[v]);
            noStroke();
            rect(x * s, y * s, s, s);

            fill(pal[(v + 3) % 6]);
            text(v % 2 ? "|" : "o", x * s + s / 2, y * s + s / 2);
        }
    }
}

```

random moves lines from right to left or left to right



```

if (frameCount % 10 === 0) {
    let y = floor(random(r));
    if (random() < 0.5) {

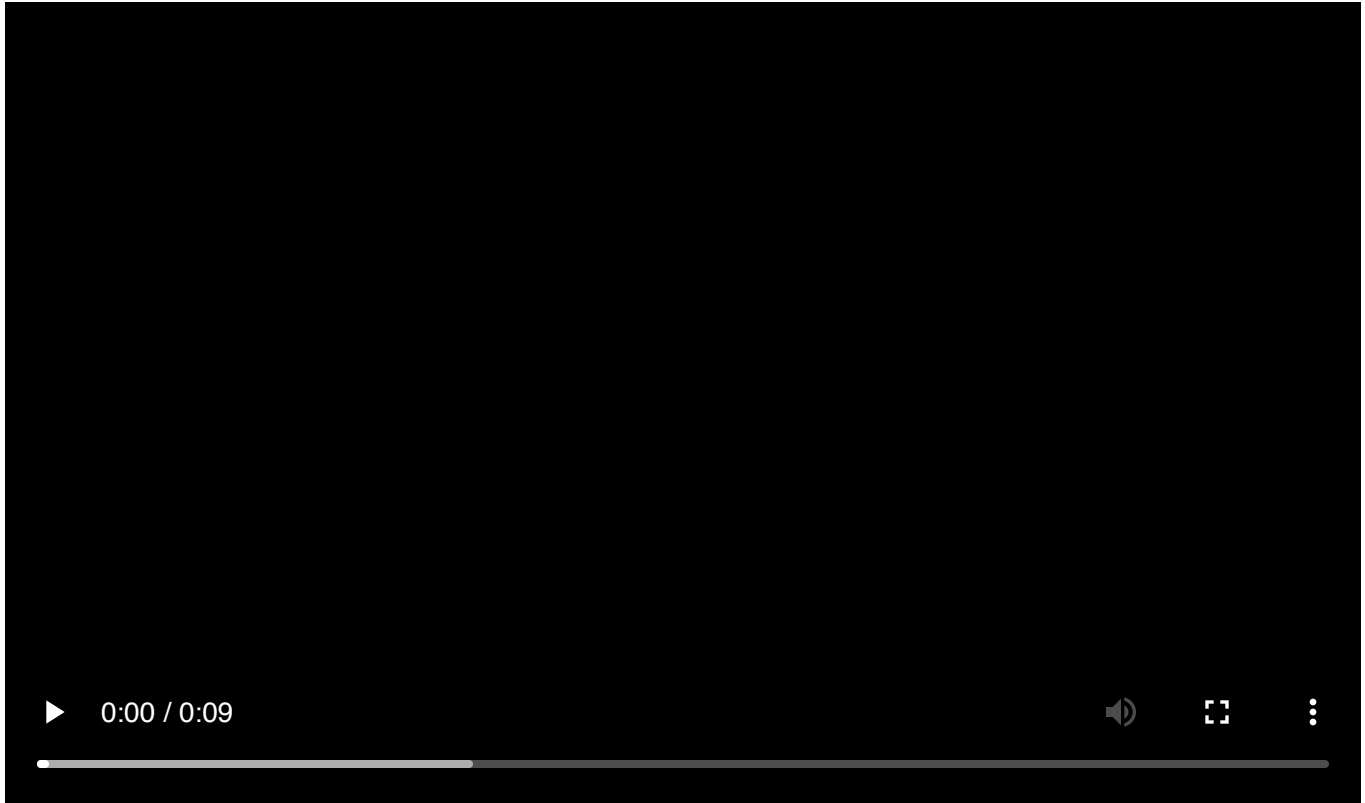
```

```

    g[y].push(g[y].shift()); // links
  } else {
    g[y].unshift(g[y].pop()); // rechts
  }
}

```

here also up and down - down and up



```

let s = 25; //cellsize
let c, r, g = []; //c=spalten, r=reihen, grid

const pal = [
  "#006432", // grün
  "#a2c72b", // hellgrün
  "#173b19", // dunkelgrün
  "#ffff00", // gelb
  "#ff0066", // pink
  "#ff00ff"  // magenta
];

function setup() {
  createCanvas(windowWidth, windowHeight);
  textAlign(CENTER, CENTER);
  textSize(s * 0.75);
  textFont("monospace");

```

```

    initGrid();
}

function draw() {
    background(0);

    patternUpdate();

    for (let y = 0; y < r; y++) { //alle reihen
        for (let x = 0; x < c; x++) { //alle spalten
            let v = g[y][x]; //wert aktuelle zelle
            let fg = (v + 3) % 6; //zeichenfarbe
            fill(pal[v]); //fill der zelle
            noStroke();
            rect(x * s, y * s, s, s);
            fill(pal[fg]); //zeichenfarbe

            text((v % 2) ? "|" : "o", x * s + s / 2, y * s + s / 2); //text in
mitte cell
        }
    }
}

function initGrid() { //erstellt grid
    c = floor(width / s); //spalten
    r = floor(height / s); //reihen
    g = []; //grid leeren

    for (let y = 0; y < r; y++) { //reihe erzeugen
        g[y] = []; //neue zeile

        for (let x = 0; x < c; x++) { //spalten
            g[y][x] = (x ^ y) % 6; //entweder oder pattern
        }
    }
}

function patternUpdate() { //bewegen
    if (frameCount % 3 !== 0 || g.length === 0) return; //jedes 3 frame
ausführen
    let k = floor(random(4)); //random action
    if (k === 0) { //zeile nach links
        let y = floor(random(r)); //random zeile

        g[y].push(g[y].shift()); //erstes element nach hinten
    }
}

```

```

if (k === 1) { //zeile nach rechts verschieben
  let y = floor(random(r)); //random zeile
  g[y].unshift(g[y].pop()); //letzte cell nach vorne
}

if (k === 2) { //spalte nach unten
  let x = floor(random(c)); //random spalte
  let t = g[r - 1][x]; //untersten wert speichern

  for (let y = r - 1; y > 0; y--) //alle nach unten kopieren
    g[y][x] = g[y - 1][x];

  g[0][x] = t; //wert einsetzen
}

if (k === 3) { //spalte nach oben
  let x = floor(random(c)); //random spalte

  let t = g[0][x]; //oberster wert speichern

  for (let y = 0; y < r - 1; y++) //copy werte
    g[y][x] = g[y + 1][x];
  g[r - 1][x] = t; // wert einsetzen
}
}

```

next idea was to color pixels black during the live performance, gradually building a city skyline. it reminded me of a city at night, with illuminated buildings and glowing windows. it feels magical and colorful, but it could also come across as artificial and maybe overstimulating

▶ 0:00 / 0:06



```
//nach render
fill(0);
noStroke();

let skyline = [
  ... Array(6).fill(10), //färbt pixel schwarz array(breite) fill(höhe)
  ... Array(4).fill(18),
  ... Array(4).fill(12),
  ... Array(6).fill(8),
  ... Array(4).fill(20),
  ... Array(6).fill(16)

  //10,10,10,20,20,20,10,10,
  //14,14,14,14,9,9,9,15,15
];

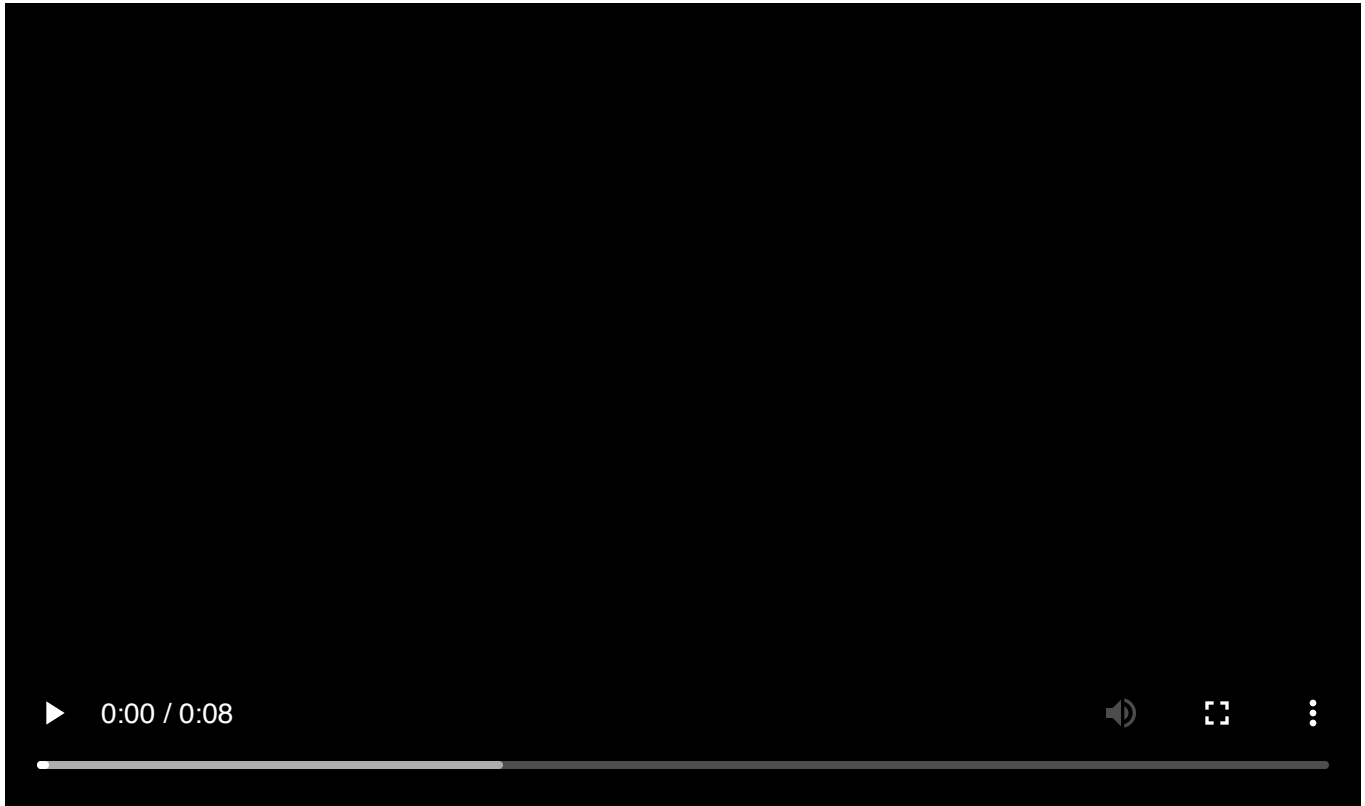
for (let i = 0; i < skyline.length; i++) {

  rect(
    i * s * 2,           // x
    0,                   // y
    s * 2,               // width
    skyline[i] * s       // height
  );
}
```



```
}  
}
```

experiment to combine organic and synthetic



```
let s = 26, g = []; //g=array für cells s=cellsize  
  
function setup() {  
  createCanvas(windowWidth, windowHeight);  
  noStroke();  
  
  for (let y = 0; y < height / s; y++) { // zeilen im grid  
    g[y] = []; // neue zeile im array  
  
    for (let x = 0; x < width / s; x++) { // spalten im grid  
      g[y][x] = random() > 0.18; // zufällig true/false  
    }  
  }  
}  
  
function draw() {  
  background(245);  
  
  for (let y = 0; y < g.length; y++) { // durch alle zeilen  
    for (let x = 0; x < g[y].length; x++) { // durch alle spalten
```

```

    if (!g[y][x]) continue; // nur aktive zellen gezeichnet

    let px = x * s + s / 2; // pixelposition x center
    let py = y * s + s / 2; // pixelposition y center

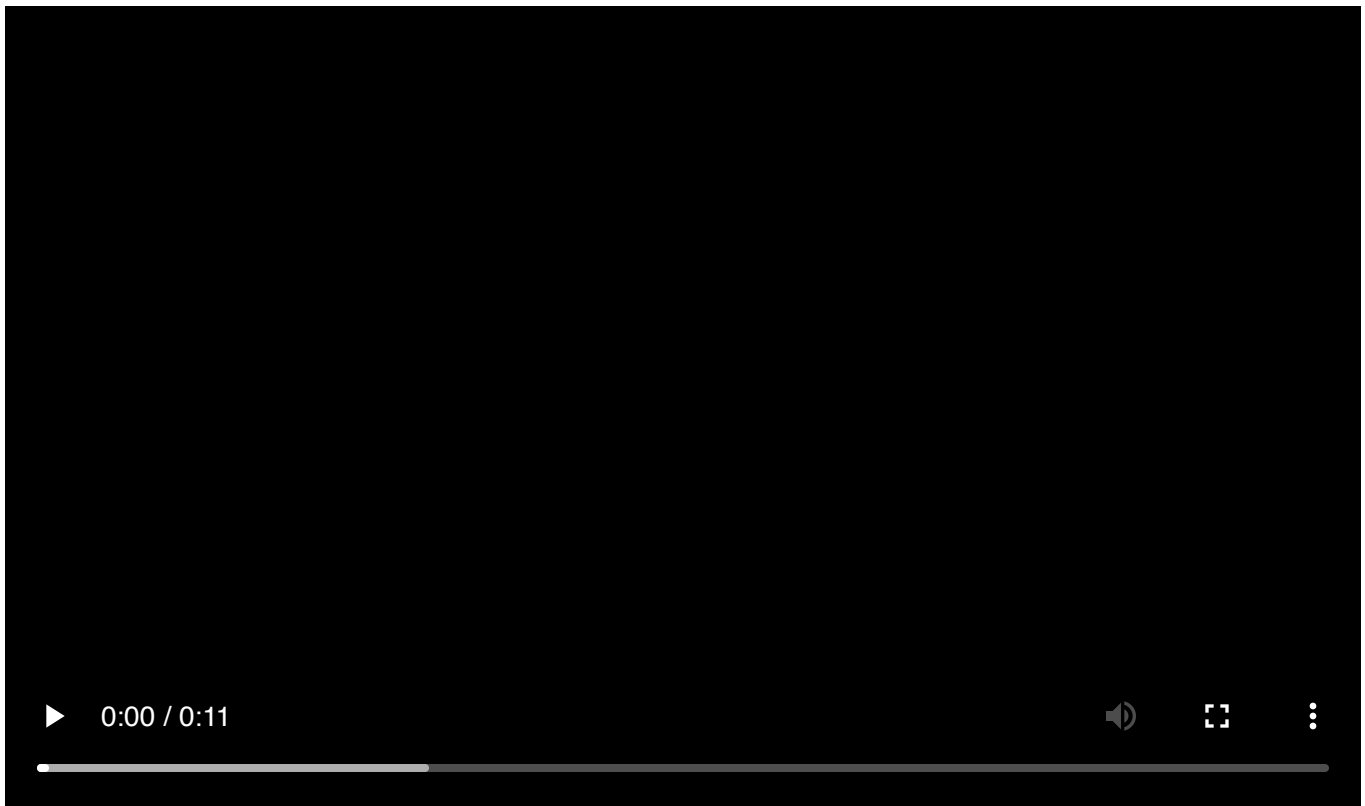
    let p = sin(frameCount * 0.03 + (x + y) * 0.4) * 0.5 + 0.5; // 0-1
welle
    let r = 4 + p * 12; // radius abhängig von puls

    fill(255, 0, 120);
    rect(x * s, y * s, s, s); //rasterzelle

    for (let i = 5; i > 0; i--) { // glow (mehrere layer)

        fill((x * 20 + i * 40) % 255, 200, 255); // farbiger verlauf
        ellipse(px, py, r * i * 0.4);
    }
  }
}
}
}

```



```

let grid = 36;
let cell;

```

```

function setup() {
  createCanvas(windowWidth, windowHeight);
  rectMode(CENTER);
  noFill();
}

function draw() {

  background(240);

  cell = width / grid; //cellsize abhängig von width

  for (let x = 0; x < grid; x++) { // schleife für x-achse
    for (let y = 0; y < grid * 1.4; y++) { // schleife für y-achse

      let px = x * cell + cell / 2; // pixel-x position der zelle -center
      let py = y * cell + cell / 2; // pixel-y position der zelle -center

      let dx = x - grid * 0.5; // abstand mitte in x richtung
      let dy = y - grid * 0.45; // abstand von verschobene mitte in y
richtung

      let shape =
        sqrt(dx * dx + dy * dy * 1.7) + // elliptische distanz
        noise(x * 0.12, y * 0.12) * 6; // perlin noise - unregelmässig

      if (shape > 11) continue; // alles außerhalb wird übersprungen

      push();
      translate(px, py);

      // zellen
      stroke(255, 90);
      fill(255, 25);
      rect(0, 0, cell * 0.96);

      // langsames pulsieren
      let energy = noise(
        x * 0.15,
        y * 0.15,
        frameCount * 0.015
      );

      // riesige blobs

```

```

let size =
  cell * 0.3 +
  energy * cell * 2.2;

if (energy > 0.7) { // noch grössere blobs
  size += cell * 2.5;
}

// ringe
for (let i = 7; i > 0; i--) {

  let s = size * (i / 7);

  drawingContext.shadowBlur = 15; // blur
  drawingContext.shadowColor = "#ff00aa"; // shadow glow

  if (i % 3 == 0) stroke("#ff0066");
  else if (i % 3 == 1) stroke("#00eeff");
  else stroke("#4400ff");

  ellipse(0, 0, s); // kreis center einer zelle
}

pop();
}
}
}

```

number pixel system with organic random movements - idea of combining both aspects

▶ 0:00 / 0:06



```
let t = 0; //zeitvariable
let s = 16; //cellsize

function setup() {
  createCanvas(windowWidth, windowHeight);
  textFont("monospace");
  textAlign(CENTER, CENTER);
  textSize(10);
  strokeWeight(0);
  //strokeWeight(1);
}

function draw() {
  //background(0)
  background( "#006432");

  for (let y = 0; y < height; y += s) { //zeile für zeile
    beginShape();

    for (let x = 0; x < width; x += s) { //spalte für spalte

      let n = noise(x * 0.004, y * 0.004, t); //perlin noise

      let flow = //wellenbewegung
        sin(x * 0.01 + t * 2) * 10 + // horizontal
        cos(y * 0.01 + t) * 5;      // vertikal
```

```

    let yy = y + map(n, 0, 1, -50, 50) + flow; //y position berechnen

    stroke(255, 35); //linienfarbe
    line(x, yy, x + s, yy);
    fill(255);
    noStroke();

    if (n > 0.5) //wenn nois grösser als
        rect(x - 5, yy - 5, 10, 10);
    fill(0); //textfarbe
    text(int(random(2)), x, yy + 1); //random 0 oder 1
}
endShape();
}

t += 0.01; //zeit langsam erhöhen
}

```

because of the circles instead of rects it appears for me more organic



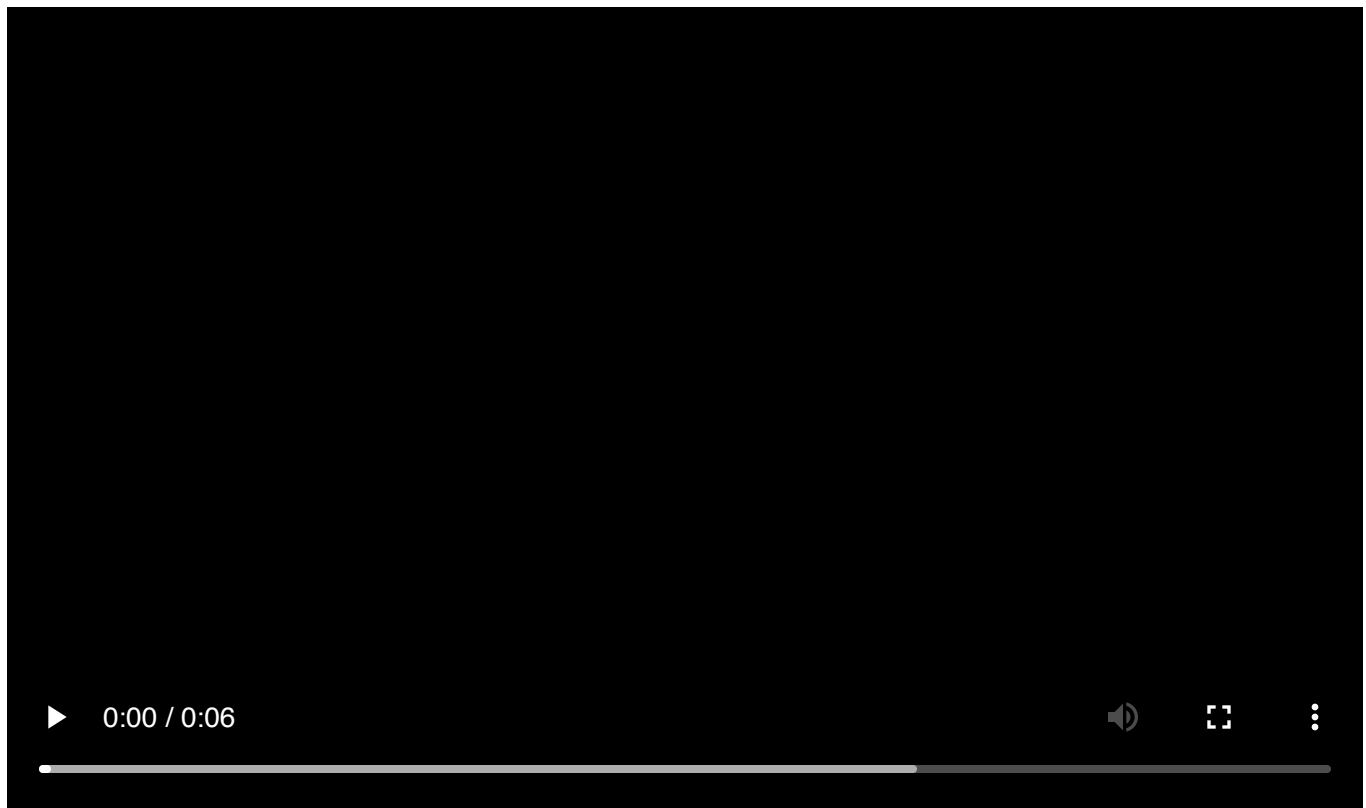
exchange with luka

he showed me his sound and we realized that it worked really well with my visuals
 we began to structure the snippets together with the sound and we also considered how the
 transitions between the different snippets could work

concept:

- something light, airy transforming to something more fluid and watery
- then pixelFlower() that aligned with the beginning of the drum section using hyrda i gradually zoom in and increase the pixelation it more o create a transition into the pattern system (skyline)
our idea was to make the snippet audio-reactive allowing the rectangles to shift to a darker green in response to louder sounds, reinforcing the heavier parts of the music
the visual would then gradually transition from dark green to lighter and more colorful tones before returning to a completely dark green state, which would serve as the transition into the flow field
- the systematic flow field would change to something more organic by unrounding the p.x and p.y
- then the last snippet would combine this two elements and be the end of the presentation

for the missing first snippet i created something windy airy with the code i already had from the flow field and just changed it a little bit



```
//perlin noise flow field  
  
//array for particles  
let particles = [];  
  
//number of particles  
const num = 2000;
```

```
//noise-scale - how "wild"
const noiseScale = 0.005/2;

function setup() {
  createCanvas(windowWidth, windowHeight);
}

function draw() {
  organicFlowField();
}

function organicFlowField() {

  //create particles only the first time - setup-ersatz
  if(particles.length === 0) {

    //loop to create vector for each particle
    for(let i = 0; i < num; i ++) {

      //random x and y positions - push inserts vectors in array
      particles.push(
        createVector(random(width), random(height))
      );
    }

    //particle color
    stroke(0);
  }

  //alpha 0.1 = only slightly overpaints the old frame
  background(255,30);

  //loop to move particles
  for(let i = 0; i < num; i ++) {

    //get particles
    let p = particles[i];
```



```

//paint particles
point(p.x, p.y);

//natural random values not like chaotic like random - 3D-noise
let n = noise(
  p.x * noiseScale,
  p.y * noiseScale,
  frameCount * 100
);

//calculate angle (TAU = 2 * PI)
let a = TAU * (n + 0.3 * noise(p.x * 0.01, p.y * 0.01));

//moves particles - *3 = speed
p.x += cos(a)*2;
p.y += sin(a)*2;

//if not on screen reset it so there are always num particles
if (p.x < 0) p.x = width;
  if (p.x > width) p.x = 0;
  if (p.y < 0) p.y = height;
  if (p.y > height) p.y = 0;
}
}

```

putting a hydra effect on top would slowly transform it to something more watery (because of the water sounds)

▶ 0:00 / 0:07



```
//perlin noise flow field

//array for particles
let particles = [];

//number of particles
const num = 1000;

//noise-scale - how "wild"
const noiseScale = 0.005/2;

let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js',
'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']

// sandbox - start
H.pixelDensity(2) // 2x = retina, set <= 1 if laggy

s0.initP5() // send p5 to hydra
P5.toggle(0) // optionally hide p5

src(s0)
  .modulate(noize())
  .out()
// sandbox - end
```

```
function setup() {  
  createCanvas(windowWidth, windowHeight);  
}
```

```
function draw() {
  organicFlowField();
}
```

```
function organicFlowField() {
```

```
//create particles only the first time - setup-ersatz
```

```
if(particles.length === 0) {
```

```
//loop to create vector for each particle
```

```
for(let i = 0; i < num; i++) {
```

```
//random x and y positions - push inserts vectors in array
```

```
particles.push(
```

```
createVector(random(width), random(height))
```

);

}

```
//particle color
```

```
stroke(0);
```

}

```
//alpha 0.1 = only slightly overpaints the old frame
```

```
background(255,0.1);
```

```
//loop to move particles
```

```
for(let i = 0; i < num; i++) {
```

```
//get particles
```

```
let p = particles[i];
```

```
//paint particles
```

```
point(p.x, p.y);
```

```
//natural random values not like chaotic like random - 3D-noise
```

```
let n = noise(
```

```

    p.x * noiseScale,
    p.y * noiseScale,
    frameCount * 100
);

//calculate angle (TAU = 2 * PI)
let a = TAU * (n + 0.3 * noise(p.x * 0.01, p.y * 0.01));

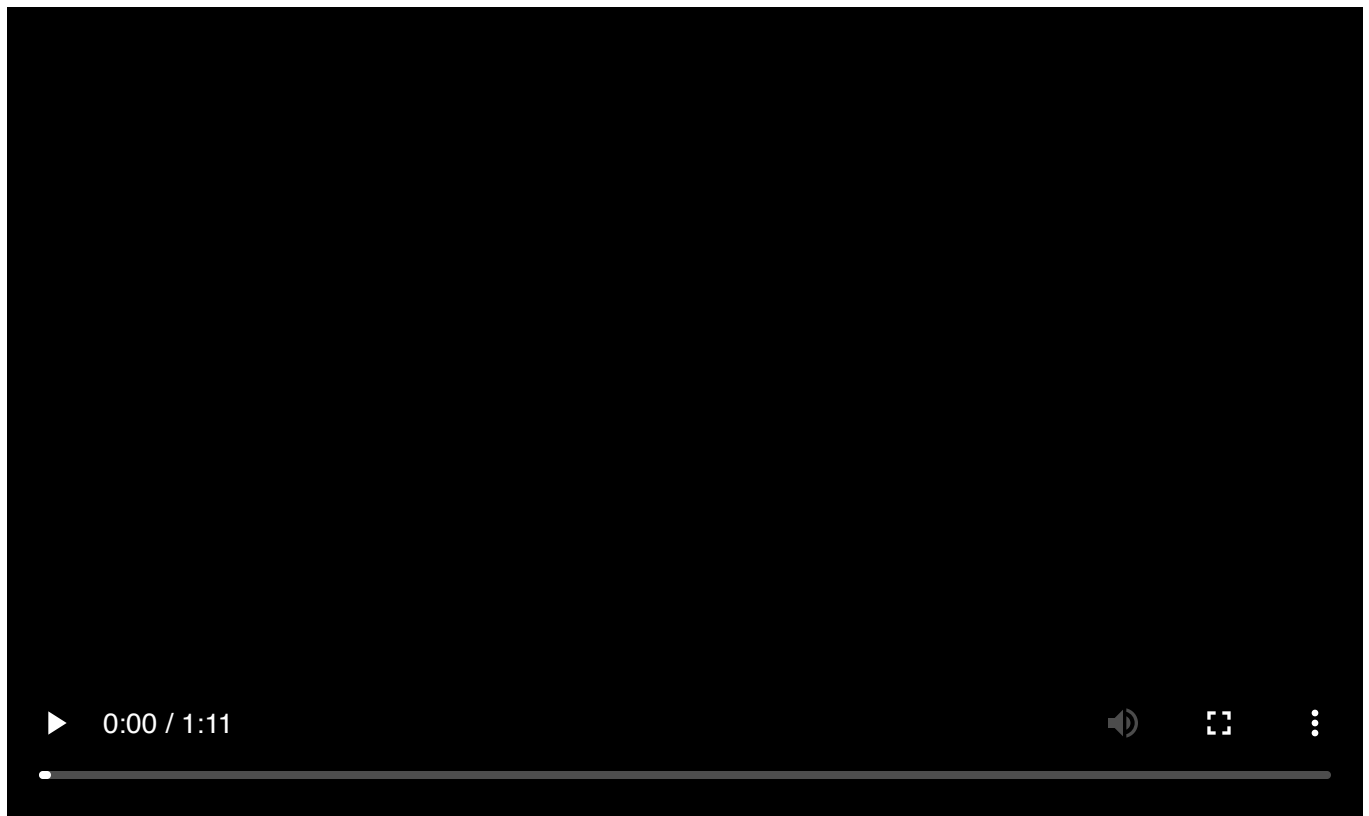
//moves particles - *3 = speed
p.x += cos(a)*2;
p.y += sin(a)*2;

//if not on screen reset it so there are always num particles
if(
  p.x < 0 ||
  p.x > width ||
  p.y < 0 ||
  p.y > height
) {
  p.x = 10;
  p.y = 10;
}
}
}

```

from this point i would change the snippet in the draw field while keeping the Hydra effect layered on top of pixelFlower()

here is the first part of the live performance



in the next step i would remove that effect and replace it with a other hydra effect
then the next effect to increase the pixelation so the transition would eventually lead into the
systematic pixel pattern

this would be the second part



unfortunately i couldn't find time to make this part audio-reactive so the change to the next snippet currently feels more like a cut than a transition

to transform the systematic flow field to something more organic i would remove the rounding of $p.x$ and $p.y$, resulting in smoother and more fluid motion

from here i would overwrite it with the last snippet that would combine the two, organic and synthetic and to merge everything together



all parts together, the finished live performance without sound would look like this

▶ 0:00 / 3:50



```
//snippet 1 and 2
let libs = ['https://unpkg.com/hydra-synth', 'includes/libs/hydra-synth.js',
'https://cdn.jsdelivr.net/gh/ffd8/hy5@main/hy5.js', 'includes/libs/hy5.js']
H.pixelDensity(2)
s0.initP5()
```

```
//snippet 3
let cellSize = 25;
let c, r, grid;
const pal = [
  "#006432",
  "#a2c72b",
  "#173b19",
  "#ffff00",
  "#ff0066",
  "#ff00ff"
];
```

```
//snippet 4
let windParticles = [];
const particleCount = 1000;
const flowNoiseScale = 0.0025;
```

```
//snippet 5
```

```

let t = 0;
let s = 16;

function setup() {
  createCanvas(windowWidth, windowHeight)
}

function draw() {
  if(frameCount%2===0){

  }

}

//snippet 1
function windField() {
  if (windParticles.length === 0) {
    for (let i = 0; i < particleCount; i++) {
      windParticles.push(
        createVector(random(width), random(height))
      );
    }
    stroke(0);
  }
  background(255, 30);
  for (let i = 0; i < particleCount; i++) {
    let p = windParticles[i];
    point(p.x, p.y);
    let n = noise(
      p.x * flowNoiseScale,
      p.y * flowNoiseScale,
      frameCount * 100
    );
    let a = TAU * (n + 0.3 * noise(p.x * 0.01, p.y * 0.01));
    p.x += cos(a) * 2;
    p.y += sin(a) * 2;
    if (p.x < 0) p.x = width;
    if (p.x > width) p.x = 0;
    if (p.y < 0) p.y = height;
    if (p.y > height) p.y = 0;
  }
}

```


//snippet 2

```
function pixelFlower() {
  background(255)
  let cellSizeLocal = 10;
  for (let x = 0; x < width; x += cellSizeLocal) {
    let d = random(-20, 20);
    for (let y = 0; y < height; y += cellSizeLocal) {
      d += random(-.4, .4);
      stroke(noise(x * .01, y * .002) > .5 ? 0 : 255);
      line(x + d - 2, y - 2, x + d + 2, y + 2);
      line(x + d - 2, y + 2, x + d + 2, y - 2);
    }
  }
  for (let k = 0; k < 6; k++)
    ast(
      width / 2 + random(-20, 20),
      height,
      -PI / 2 + random(-.2, .2),
      90
    );
  function ast(x, y, a, n) {
    for (let i = 0; i < n; i++) {
      fill(random(["#006432", "#a2c72b", "#173b19"]));
      noStroke();
      rect(
        round(x / cellSizeLocal) * cellSizeLocal,
        round(y / cellSizeLocal) * cellSizeLocal,
        cellSizeLocal,
        cellSizeLocal
      );
      if (random() < .02)
        ast(
          x,
          y,
          a + random(-.8, .8),
          n * .6
        );
      x += cos(a) * cellSizeLocal;
      y += sin(a) * cellSizeLocal;
      a += random(-.12, .12);
    }
    for (let i = 0; i < 500; i++) {
      fill(random(["#ffff00", "#f06", "#f0f"]));
      rect(
        round((x + randomGaussian() * 35) / cellSizeLocal) * cellSizeLocal,
```

```

        round((y + randomGaussian() * 35) / cellSizeLocal) * cellSizeLocal,
        cellSizeLocal,
        cellSizeLocal
    );
}
}
}

```

//snippet 3

```

function pattern() {
    if (!pattern.g) {
        pattern.c = floor(windowWidth / cellSize);
        pattern.r = floor(windowHeight / cellSize);
        pattern.g = [];
        for (let y = 0; y < pattern.r; y++) {
            pattern.g[y] = [];
            for (let x = 0; x < pattern.c; x++) {
                pattern.g[y][x] = (x ^ y) % 6;
            }
        }
    }
    let colCount = pattern.c;
    let rowCount = pattern.r;
    let grid = pattern.g;
    background(0);
    if (frameCount % 3 === 0) {
        let k = floor(random(4));
        if (k === 0) {
            let y = floor(random(rowCount));
            grid[y].push(grid[y].shift());
        }
        if (k === 1) {
            let y = floor(random(rowCount));
            grid[y].unshift(grid[y].pop());
        }
        if (k === 2) {
            let x = floor(random(colCount));
            let t = grid[rowCount - 1][x];
            for (let y = rowCount - 1; y > 0; y--) {
                grid[y][x] = grid[y - 1][x];
            }
            grid[0][x] = t;
        }
        if (k === 3) {
            let x = floor(random(colCount));
            let t = grid[0][x];

```

```

    for (let y = 0; y < rowCount - 1; y++) {
      grid[y][x] = grid[y + 1][x];
    }
    grid[rowCount - 1][x] = t;
  }
}
textAlign(CENTER, CENTER);
textSize(cellSize * 0.75);
textFont("monospace");
for (let y = 0; y < rowCount; y++) {
  for (let x = 0; x < colCount; x++) {
    let v = grid[y][x];
    let fg = (v + 3) % 6;
    fill(pal[v]);
    noStroke();
    rect(x * cellSize, y * cellSize, cellSize, cellSize);
    fill(pal[fg]);
    text((v % 2) ? "|" : "o",
      x * cellSize + cellSize / 2,
      y * cellSize + cellSize / 2
    );
  }
}
}
}

```

//snippet 4

```

function organicFlowField() {
  if (windParticles.length === 0) {
    for (let i = 0; i < particleCount; i++) {
      windParticles.push(
        createVector(random(width), random(height))
      );
    }
    stroke(255, 255, 0);
  }
  background(0,100,50,0.1);
  for (let i = 0; i < particleCount; i++) {
    let p = windParticles[i];
    point(p.x, p.y);
    let n = noise(
      p.x * flowNoiseScale,
      p.y * flowNoiseScale,
      frameCount * flowNoiseScale * flowNoiseScale
    );
    let a = TAU * n;
  }
}

```

```

p.x += round(cos(a) * 2);
p.y += round(sin(a) * 2);
if (
  p.x < 0 ||
  p.x > width ||
  p.y < 0 ||
  p.y > height
) {
  p.x = 10;
  p.y = 10;
}
}
}

```

//snippet 5

```

function combinationField() {

  textFont("monospace");
  textAlign(CENTER, CENTER);
  textSize(10);
  strokeWeight(0);

  background("#006432");

  for (let y = 0; y < height; y += s) {
    for (let x = 0; x < width; x += s) {

      let n = noise(x * 0.004, y * 0.004, t);

      let flow =
        sin(x * 0.01 + t * 2) * 10 +
        cos(y * 0.01 + t) * 5;

      let yy = y + map(n, 0, 1, -50, 50) + flow;

      stroke(255, 35);
      line(x, yy, x + s, yy);

      noStroke();
      fill(255);

      if (n > 0.5) {
        rect(x - 5, yy - 5, 8, 8, 10);
      }

      fill(0);
    }
  }
}

```

```
    text(int(random(2)), x, yy + 1);  
  }  
}  
  
t += 0.01; // Zeit jetzt hier  
}
```

i know some transitions are not smooth, the one from pattern() to organicFlowField() is missing the audio-reactive part and also the last transition from organicFlowField to combinationField() is not the best

for the live coding I changed the settings of p5LIVE so the picture only reloads if I press ctrl+enter, so the visual don't disappear if I change the code